electricity-price-forecasting-with-dnns-eda

November 10, 2026

1 Electricity Price Forecasting with Deep Neural Networks

Electricity is a *basic human need* and definitely one of the most important factors of societal progress. In recent decades however, electricity has entered the market as a tradeable commodity and the power industry of many countries has been **deregulated**. In Spain, the Electric Power Act 54/1997 exposed all of the stakeholders to **high amounts of uncertainty** as the price of electricity is determined by countless factors and also, due to the fact that electricity cannot be stored in large quantities efficiently [1]. With the emergence of this new market, the need for reliable forecasting methods at all scales (hourly, daily, long-term, etc.) has also emerged and has become a large area of research.

The goal of this kernel is to compare different **deep neural network architectures** (+XGBoost) on the task of **predicting the next hour's electricity price** by using the past values of the electricity price as well as those of another features related to energy generation and weather conditions. Furthermore, the kernel contains a meticulous *exploration and cleaning* of the data, *time series analysis* of the electricity price and careful *feature engineering*. With further research and development (e.g. as a forecasting model which is updated in real-time) a similar approach could possibly prove useful to all the stakeholders (electric power companies, investors, etc.) involved in energy markets. For the time being, I believe that this kernel can serve as a **future reference** for aspiring data scientists, as it contains an *end-to-end time series forecasting project* and demonstrates the -minimum- level of immersion that a dataset needs in order to turn into something which is actually applicable in the real world.

In the **original project**, which was conducted for a postgraduate course, I compared the performance (using the Root Mean Squared Error as the performance metric) of 5 different architectures (LSTM, stacked LSTM, CNN, CNN-LSTM and Time Distributed MLP) for both univariate and multivariate forecasting (i.e. using only the previous time-steps of the electricity price vs. also using other features) using a different number of previous time-steps as the features for the models (3, 10 and 25 previous time-steps for all the used features). In **this particular kernel**, you will find an application of all the aforementioned deep learning architecures, as well as two more approaches: the *Encoder-Decoder* architecure and the *XGBoost regressor*. Furthermore, in all these applications, I use the 25 previous time-steps of all the features (multivariate forecasting) that have been extracted or generated, after applying *PCA (Principal Component Analysis)*. The **Adam optimizer** is used in all the deep learning architectures and, in order to choose its learning rate, I originally conducted preliminary tests using the the *learning rate scheduler* callback starting from a learning rate equal to 0.0001 and gradually increasing it by a factor of 10 every 10 epochs; a step which is omitted in this kernel.

2 Table of Contents

- 1. Exploration and Cleaning
  - 1.1. Energy dataset
  - 1.2. Weather features dataset
  - 1.3. Merging the two datasets
- 2. Visualizations and Time Series Analysis

  - 2.1. Useful visualizations and insights
  - 2.2. Decomposition and stationarity tests
  - 2.3. Autocorrelation, partial autocorrelation and cross-correlation
- 3. Feature Engineering
  - 3.1. Feature generation
  - 3.2. Feature selection
- 4. Electricity Price Forecasting
  - 4.1. Naive forecast
  - 4.2. XGBoost
  - 4.3. LSTM
  - 4.4. Stacked LSTMs
  - 4.5. CNN
  - 4.6. CNN-LSTM
  - 4.7. Time Disitributed MLP
  - 4.8. Encoder-Decoder
- 5. Results
- 6. References

[1]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import tensorflow as tf
import xgboost as xgb
import os
import warnings
from tensorflow.keras.layers import Dense, LSTM, Conv1D, MaxPooling1D, TimeDistributed, Flatten, Dropout,␣
 ↪RepeatVector
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller, kpss, ccf
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from math import sqrt

%matplotlib inline

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

```
/kaggle/input/energy-consumption-generation-prices-and-
weather/energy_dataset.csv
/kaggle/input/energy-consumption-generation-prices-and-
weather/weather_features.csv
```

[2]:
```
warnings.simplefilter(action='ignore', category=(FutureWarning, UserWarning))
```

3 1. Exploration and Cleaning

In this dataset, we have two .csv files which contain hourly information about the electricity generation and weather in Spain for the period 2015-2019 (4 years). In particular:

> **`'weather_features.csv'`:** Contains hourly information about the weather conditions (e.g. temperature, wind speed, humidity, rainfall, qualitative desctiption) of 5 major cities in Spain (Madrid, Barcelona, Valencia, Seville and Bilbao).
>
> **`'energy_dataset.csv'`:** Contains hourly information about the generation of energy in Spain. In particular, there is info (in MW) about the amount of electricty generated by the various energy sources (fossil gas, fossil hard coal and wind energy dominate the energy grid), as well as about the total load (energy demand) of the national grid and the price of energy (€/MWh). *Note: Since the generation of each energy type is in MW and the time-series contains hourly info, the value of each cell represents MWh (Megawatt hours).*

The information that we have about the weather of 5 major cities in Spain (highlighted by a red star on the map below) is probably more than enough for our analysis, since their geographic distribution covers most of the part of Spain's territory in a uniform manner. Moreover, it is useful to note that these 5 cities alone comprise approximately 1/3rd of the total population of Spain.



[3]:
```
# Read the datasets

df_weather = pd.read_csv(
    '/kaggle/input/energy-consumption-generation-prices-and-weather/weather_features.csv',
    parse_dates=['dt_iso']
)

df_energy = pd.read_csv(
```

```
    '/kaggle/input/energy-consumption-generation-prices-and-weather/energy_dataset.csv',
    parse_dates=['time']
)
```

3.1 1.1. Energy dataset

[4]:
```
df_energy.head()
```

[4]:
```
                       time  generation biomass  \
0  2015-01-01 00:00:00+01:00               447.0
1  2015-01-01 01:00:00+01:00               449.0
2  2015-01-01 02:00:00+01:00               448.0
3  2015-01-01 03:00:00+01:00               438.0
4  2015-01-01 04:00:00+01:00               428.0

   generation fossil brown coal/lignite  generation fossil coal-derived gas  \
0                                 329.0                                0.0
1                                 328.0                                0.0
2                                 323.0                                0.0
3                                 254.0                                0.0
4                                 187.0                                0.0

   generation fossil gas  generation fossil hard coal  generation fossil oil  \
0                 4844.0                       4821.0                  162.0
1                 5196.0                       4755.0                  158.0
2                 4857.0                       4581.0                  157.0
3                 4314.0                       4131.0                  160.0
4                 4130.0                       3840.0                  156.0

   generation fossil oil shale  generation fossil peat  generation geothermal  \
0                          0.0                     0.0                    0.0
1                          0.0                     0.0                    0.0
2                          0.0                     0.0                    0.0
3                          0.0                     0.0                    0.0
4                          0.0                     0.0                    0.0

   …  generation waste  generation wind offshore  generation wind onshore  \
0  …             196.0                       0.0                   6378.0
1  …             195.0                       0.0                   5890.0
2  …             196.0                       0.0                   5461.0
3  …             191.0                       0.0                   5238.0
4  …             189.0                       0.0                   4935.0

   forecast solar day ahead  forecast wind offshore eday ahead  \
0                      17.0                                NaN
1                      16.0                                NaN
2                       8.0                                NaN
3                       2.0                                NaN
4                       9.0                                NaN

   forecast wind onshore day ahead  total load forecast  total load actual  \
0                           6436.0              26118.0            25385.0
1                           5856.0              24934.0            24382.0
2                           5454.0              23515.0            22734.0
3                           5151.0              22642.0            21286.0
4                           4861.0              21785.0            20264.0

   price day ahead  price actual
0            50.10         65.41
1            48.10         64.92
2            47.33         64.48
3            42.27         59.32
4            38.41         56.04

[5 rows x 29 columns]
```

We will drop all the columns that are constituted by zeroes and NaNs, as they are unusable. We will also remove the columns which will not be used at all in our analysis and which contain day-ahead forecasts for the total load, the solar energy and the wind energy.

[5]:
```
# Drop unusable columns

df_energy = df_energy.drop(['generation fossil coal-derived gas','generation fossil oil shale',
                            'generation fossil peat', 'generation geothermal',
                            'generation hydro pumped storage aggregated', 'generation marine',
                            'generation wind offshore', 'forecast wind offshore eday ahead',
                            'total load forecast', 'forecast solar day ahead',
                            'forecast wind onshore day ahead'],
                            axis=1)
```

[6]:
```
df_energy.describe().round(2)
```

[6]:

	generation biomass	generation fossil brown coal/lignite	\
count	35045.00	35046.00	
mean	383.51	448.06	
std	85.35	354.57	
min	0.00	0.00	
25%	333.00	0.00	
50%	367.00	509.00	
75%	433.00	757.00	
max	592.00	999.00	

	generation fossil gas	generation fossil hard coal	\
count	35046.00	35046.00	
mean	5622.74	4256.07	
std	2201.83	1961.60	
min	0.00	0.00	
25%	4126.00	2527.00	
50%	4969.00	4474.00	
75%	6429.00	5838.75	
max	20034.00	8359.00	

	generation fossil oil	generation hydro pumped storage consumption	\
count	35045.00	35045.00	
mean	298.32	475.58	
std	52.52	792.41	
min	0.00	0.00	
25%	263.00	0.00	
50%	300.00	68.00	
75%	330.00	616.00	
max	449.00	4523.00	

	generation hydro run-of-river and poundage	\
count	35045.00	
mean	972.12	
std	400.78	
min	0.00	
25%	637.00	
50%	906.00	
75%	1250.00	
max	2000.00	

	generation hydro water reservoir	generation nuclear	generation other	\
count	35046.00	35047.00	35046.00	
mean	2605.11	6263.91	60.23	
std	1835.20	839.67	20.24	
min	0.00	0.00	0.00	
25%	1077.25	5760.00	53.00	
50%	2164.00	6566.00	57.00	
75%	3757.00	7025.00	80.00	
max	9728.00	7117.00	106.00	

```
       generation other renewable  generation solar  generation waste  \
count                     35046.00          35046.00          35045.00
mean                         85.64           1432.67            269.45
std                          14.08           1680.12             50.20
min                           0.00              0.00              0.00
25%                          73.00             71.00            240.00
50%                          88.00            616.00            279.00
75%                          97.00           2578.00            310.00
max                         119.00           5792.00            357.00

       generation wind onshore  total load actual  price day ahead  \
count                 35046.00           35028.00         35064.00
mean                   5464.48           28696.94            49.87
std                    3213.69            4574.99            14.62
min                       0.00           18041.00             2.06
25%                    2933.00           24807.75            41.49
50%                    4849.00           28901.00            50.52
75%                    7398.00           32192.00            60.53
max                   17436.00           41015.00           101.99

       price actual
count      35064.00
mean          57.88
std           14.20
min            9.33
25%           49.35
50%           58.02
75%           68.01
max          116.80
```

[7]:
```
df_energy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35064 entries, 0 to 35063
Data columns (total 18 columns):
time                                         35064 non-null object
generation biomass                           35045 non-null float64
generation fossil brown coal/lignite         35046 non-null float64
generation fossil gas                        35046 non-null float64
generation fossil hard coal                  35046 non-null float64
generation fossil oil                        35045 non-null float64
generation hydro pumped storage consumption  35045 non-null float64
generation hydro run-of-river and poundage   35045 non-null float64
generation hydro water reservoir             35046 non-null float64
generation nuclear                           35047 non-null float64
generation other                             35046 non-null float64
generation other renewable                   35046 non-null float64
generation solar                             35046 non-null float64
generation waste                             35045 non-null float64
generation wind onshore                      35046 non-null float64
total load actual                            35028 non-null float64
price day ahead                              35064 non-null float64
price actual                                 35064 non-null float64
dtypes: float64(17), object(1)
memory usage: 4.8+ MB
```

The 'time' column, which we also want to function as the index of the observations in a time-series, has not been parsed correctly and is recognized as an object.

[8]:
```
# Convert time to datetime object and set it as index

df_energy['time'] = pd.to_datetime(df_energy['time'], utc=True, infer_datetime_format=True)
df_energy = df_energy.set_index('time')
```

[9]:
```
# Find NaNs and duplicates in df_energy

print('There are {} missing values or NaNs in df_energy.'
      .format(df_energy.isnull().values.sum()))

temp_energy = df_energy.duplicated(keep='first').sum()

print('There are {} duplicate rows in df_energy based on all columns.'
      .format(temp_energy))
```

```
There are 292 missing values or NaNs in df_energy.
There are 0 duplicate rows in df_energy based on all columns.
```

As we can see, `df_energy` has no duplicate values. Nevertheless, it has some NaNs and thus, we have to investigate further. Since this is a time-series forecasting task, we cannot simply drop the rows with the missing values and it would be a better idea to fill the missing values using interpolation.

[10]:
```
# Find the number of NaNs in each column

df_energy.isnull().sum(axis=0)
```

[10]:
```
generation biomass                            19
generation fossil brown coal/lignite          18
generation fossil gas                         18
generation fossil hard coal                   18
generation fossil oil                         19
generation hydro pumped storage consumption   19
generation hydro run-of-river and poundage    19
generation hydro water reservoir              18
generation nuclear                            17
generation other                              18
generation other renewable                    18
generation solar                              18
generation waste                              19
generation wind onshore                       18
total load actual                             36
price day ahead                                0
price actual                                   0
dtype: int64
```

Most null values can be found in the 'total load actual' column. Therefore, it is a good idea to visualize it and see what we can do. The good news is that there are no NaNs in the 'price actual' column, which we will use as the target variable in order to train our model. The similar numbers in null values in the columns which have to do with the type of energy generation *probably indicate that they will also appear in the same rows*. Let us first define a plot function which we will then use so as to visualize the 'total load actual' column, as well as other columns.

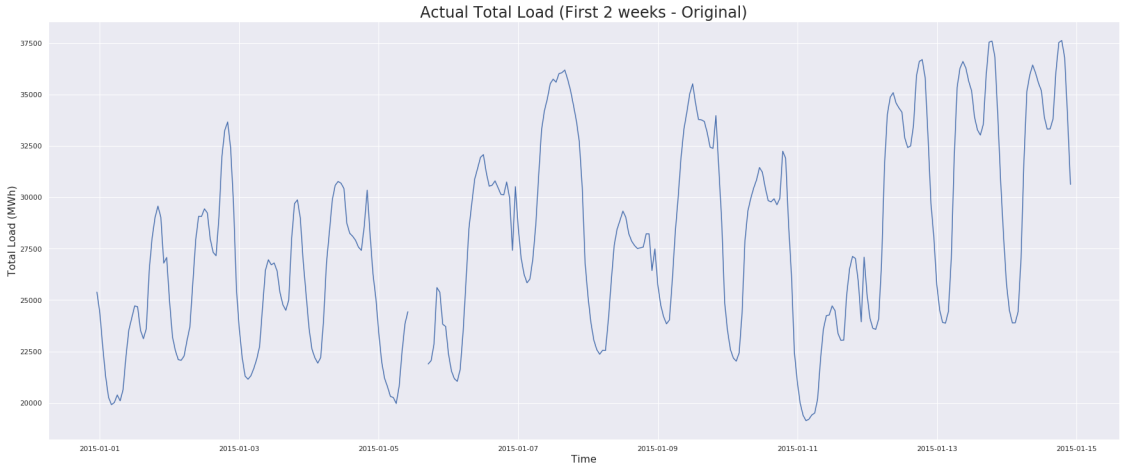
[11]:
```
# Define a function to plot different types of time-series

def plot_series(df=None, column=None, series=pd.Series([]),
                label=None, ylabel=None, title=None, start=0, end=None):
    """
    Plots a certain time-series which has either been loaded in a dataframe
    and which constitutes one of its columns or it a custom pandas series
    created by the user. The user can define either the 'df' and the 'column'
    or the 'series' and additionally, can also define the 'label', the
    'ylabel', the 'title', the 'start' and the 'end' of the plot.
    """
    sns.set()
    fig, ax = plt.subplots(figsize=(30, 12))
    ax.set_xlabel('Time', fontsize=16)
    if column:
        ax.plot(df[column][start:end], label=label)
        ax.set_ylabel(ylabel, fontsize=16)
    if series.any():
```

```
        ax.plot(series, label=label)
        ax.set_ylabel(ylabel, fontsize=16)
    if label:
        ax.legend(fontsize=16)
    if title:
        ax.set_title(title, fontsize=24)
    ax.grid(True)
    return ax
```

[12]:
```
# Zoom into the plot of the hourly (actual) total load

ax = plot_series(df=df_energy, column='total load actual', ylabel='Total Load (MWh)',
                 title='Actual Total Load (First 2 weeks - Original)', end=24*7*2)
plt.show()
```



After zooming into the first 2 weeks of the 'total load actual' column, we can already see that there are null values for a few hours. However, the number of the missing values and the behavior of the series indicate that an interpolation would fill the NaNs quite well. Let us further investigate if the null values coincide across the different columns. Let us display the last five rows.

[13]:
```
# Display the rows with null values

df_energy[df_energy.isnull().any(axis=1)].tail()
```

[13]:
```
                           generation biomass  \
time
2016-11-23 03:00:00+00:00                NaN
2017-11-14 11:00:00+00:00                0.0
2017-11-14 18:00:00+00:00                0.0
2018-06-11 16:00:00+00:00              331.0
2018-07-11 07:00:00+00:00                NaN

                           generation fossil brown coal/lignite  \
time
2016-11-23 03:00:00+00:00                                 900.0
2017-11-14 11:00:00+00:00                                   0.0
2017-11-14 18:00:00+00:00                                   0.0
2018-06-11 16:00:00+00:00                                 506.0
2018-07-11 07:00:00+00:00                                   NaN

                           generation fossil gas  generation fossil hard coal  \
time
2016-11-23 03:00:00+00:00                 4838.0                       4547.0
```

```
2017-11-14 11:00:00+00:00               10064.0                         0.0
2017-11-14 18:00:00+00:00               12336.0                         0.0
2018-06-11 16:00:00+00:00                7538.0                      5360.0
2018-07-11 07:00:00+00:00                   NaN                         NaN

                           generation fossil oil  \
time
2016-11-23 03:00:00+00:00                 269.0
2017-11-14 11:00:00+00:00                   0.0
2017-11-14 18:00:00+00:00                   0.0
2018-06-11 16:00:00+00:00                 300.0
2018-07-11 07:00:00+00:00                   NaN

                           generation hydro pumped storage consumption  \
time
2016-11-23 03:00:00+00:00                                       1413.0
2017-11-14 11:00:00+00:00                                          0.0
2017-11-14 18:00:00+00:00                                          0.0
2018-06-11 16:00:00+00:00                                          1.0
2018-07-11 07:00:00+00:00                                          NaN

                           generation hydro run-of-river and poundage  \
time
2016-11-23 03:00:00+00:00                                       795.0
2017-11-14 11:00:00+00:00                                         0.0
2017-11-14 18:00:00+00:00                                         0.0
2018-06-11 16:00:00+00:00                                      1134.0
2018-07-11 07:00:00+00:00                                         NaN

                           generation hydro water reservoir  \
time
2016-11-23 03:00:00+00:00                            435.0
2017-11-14 11:00:00+00:00                              0.0
2017-11-14 18:00:00+00:00                              0.0
2018-06-11 16:00:00+00:00                           4258.0
2018-07-11 07:00:00+00:00                              NaN

                           generation nuclear  generation other  \
time
2016-11-23 03:00:00+00:00              5040.0              60.0
2017-11-14 11:00:00+00:00                 0.0               0.0
2017-11-14 18:00:00+00:00                 0.0               0.0
2018-06-11 16:00:00+00:00              5856.0              52.0
2018-07-11 07:00:00+00:00                 NaN               NaN

                           generation other renewable  generation solar  \
time
2016-11-23 03:00:00+00:00                        85.0              15.0
2017-11-14 11:00:00+00:00                         0.0               0.0
2017-11-14 18:00:00+00:00                         0.0               0.0
2018-06-11 16:00:00+00:00                        96.0             170.0
2018-07-11 07:00:00+00:00                         NaN               NaN

                           generation waste  generation wind onshore  \
time
2016-11-23 03:00:00+00:00             227.0                   4598.0
2017-11-14 11:00:00+00:00               0.0                      0.0
2017-11-14 18:00:00+00:00               0.0                      0.0
2018-06-11 16:00:00+00:00             269.0                   9165.0
2018-07-11 07:00:00+00:00               NaN                      NaN

                           total load actual  price day ahead  price actual
time
2016-11-23 03:00:00+00:00           23112.0            43.19         49.11
2017-11-14 11:00:00+00:00               NaN            60.53         66.17
2017-11-14 18:00:00+00:00               NaN            68.05         75.45
2018-06-11 16:00:00+00:00               NaN            69.87         64.93
```

```
2018-07-11 07:00:00+00:00                NaN            63.01         69.79
```

If we manually searched through all of them, we would confirm that the null values in the columns which have to do with the type of energy generation mostly coincide. The null values in 'actual total load' also coincide with the aforementioned columns, but also appear in other rows as well. In order to handle the null values in `df_energy`, we will use a linear interpolation with a forward direction. Perhaps other kinds of interpolation would be better; nevertheless, we prefer to use the simplest model possible. Only a small part of our input data will be noisy and it will not affect performance noticeably.

[14]:
```
# Fill null values using interpolation

df_energy.interpolate(method='linear', limit_direction='forward', inplace=True, axis=0)
```

[15]:
```
# Display the number of non-zero values in each column

print('Non-zero values in each column:\n', df_energy.astype(bool).sum(axis=0), sep='\n')
```

```
Non-zero values in each column:

generation biomass                            35060
generation fossil brown coal/lignite          24540
generation fossil gas                         35063
generation fossil hard coal                   35061
generation fossil oil                         35061
generation hydro pumped storage consumption   22450
generation hydro run-of-river and poundage    35061
generation hydro water reservoir              35061
generation nuclear                            35061
generation other                              35060
generation other renewable                    35061
generation solar                              35061
generation waste                              35061
generation wind onshore                       35061
total load actual                             35064
price day ahead                               35064
price actual                                  35064
dtype: int64
```

It look like `df_energy` has been cleaned successfully and is ready for further use as input into our model. The 1-4 zeroes in the columns which have to do with energy generation by type should not concern us very much. The 'generation hydro pumped storage consumption' may look suspicious, but we should have in mind that this type of energy is only used for load balancing, being consumed when in peak energy demands.

3.2 1.2. Weather features dataset

[16]:
```
df_weather.head()
```

[16]:
```
                     dt_iso city_name     temp  temp_min  temp_max  pressure  \
0  2015-01-01 00:00:00+01:00  Valencia  270.475   270.475   270.475      1001
1  2015-01-01 01:00:00+01:00  Valencia  270.475   270.475   270.475      1001
2  2015-01-01 02:00:00+01:00  Valencia  269.686   269.686   269.686      1002
3  2015-01-01 03:00:00+01:00  Valencia  269.686   269.686   269.686      1002
4  2015-01-01 04:00:00+01:00  Valencia  269.686   269.686   269.686      1002

   humidity  wind_speed  wind_deg  rain_1h  rain_3h  snow_3h  clouds_all  \
0        77           1        62      0.0      0.0      0.0           0
1        77           1        62      0.0      0.0      0.0           0
2        78           0        23      0.0      0.0      0.0           0
3        78           0        23      0.0      0.0      0.0           0
4        78           0        23      0.0      0.0      0.0           0

   weather_id weather_main weather_description weather_icon
0         800        clear        sky is clear          01n
```

```
1         800        clear        sky is clear          01n
2         800        clear        sky is clear          01n
3         800        clear        sky is clear          01n
4         800        clear        sky is clear          01n
```

[17]:
```
df_weather.describe().round(2)
```

[17]:
```
             temp   temp_min   temp_max    pressure   humidity  wind_speed  \
count  178396.00  178396.00  178396.00   178396.00  178396.00   178396.00
mean      289.62     288.33     291.09     1069.26      68.42        2.47
std         8.03       7.96       8.61     5969.63      21.90        2.10
min       262.24     262.24     262.24        0.00       0.00        0.00
25%       283.67     282.48     284.65     1013.00      53.00        1.00
50%       289.15     288.15     290.15     1018.00      72.00        2.00
75%       295.15     293.73     297.15     1022.00      87.00        4.00
max       315.60     315.15     321.15  1008371.00     100.00      133.00

         wind_deg    rain_1h    rain_3h    snow_3h  clouds_all  weather_id
count  178396.00  178396.00  178396.00  178396.00   178396.00   178396.00
mean      166.59       0.08       0.00       0.00       25.07      759.83
std       116.61       0.40       0.01       0.22       30.77      108.73
min         0.00       0.00       0.00       0.00        0.00      200.00
25%        55.00       0.00       0.00       0.00        0.00      800.00
50%       177.00       0.00       0.00       0.00       20.00      800.00
75%       270.00       0.00       0.00       0.00       40.00      801.00
max       360.00      12.00       2.32      21.50      100.00      804.00
```

Here, we can see that all columns of `df_weather` have the same number of rows; we still have to check what is the case for each city individually, though. We should note that the temperatures are in Kelvin. The most important thing to notice is that there are some **problems** and **outliers**. In particular: - There is at least one outlier in the 'pressure' column as the maximum value is 1,008,371 hPa or approximately 100 MPa, which is roughly the pressure at the bottom of Mariana Trench about 11 km below ocean surface [2]. This cannot be the case here. - There is at least one outlier in the 'wind_speed' column as the maximum value is 133 m/s. This measurement is close to the fastest wind speed ever recorded on Earth, caused by the 1999 Bridge Creek–Moore tornado [3], a F5 (the largest intensity of the Fujita scale) tornado [4]. A tornado of such intensity has not been recorded in Spain [5] and hopefully will not happen in the future as well. - The 'rain_3h' column is supposed to provide information about the precipitation (i.e. rain) of the last 3 hours in mm. Since the 'rain_1h' column is supposed to provide the same information but about just the last hour, it would be logical to assume that its mean would be less than that of 'rain_3h'. However, this is not the case in the statistical description above. So, it would be a good idea to further examine those columns.

[18]:
```
# Print the type of each variable in df_weather

df_weather.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178396 entries, 0 to 178395
Data columns (total 17 columns):
dt_iso                178396 non-null object
city_name             178396 non-null object
temp                  178396 non-null float64
temp_min              178396 non-null float64
temp_max              178396 non-null float64
pressure              178396 non-null int64
humidity              178396 non-null int64
wind_speed            178396 non-null int64
wind_deg              178396 non-null int64
rain_1h               178396 non-null float64
rain_3h               178396 non-null float64
snow_3h               178396 non-null float64
clouds_all            178396 non-null int64
weather_id            178396 non-null int64
weather_main          178396 non-null object
```

```
weather_description    178396 non-null object
weather_icon           178396 non-null object
dtypes: float64(6), int64(6), object(5)
memory usage: 23.1+ MB
```

We have to change the type of some of the columns, so that all of them are float64. We also have to parse 'dt_iso' correctly and actually rename it as 'time' so that it matches with the index of `df_energy`.

[19]:
```
def df_convert_dtypes(df, convert_from, convert_to):
    cols = df.select_dtypes(include=[convert_from]).columns
    for col in cols:
        df[col] = df[col].values.astype(convert_to)
    return df
```

[20]:
```
# Convert columns with int64 type values to float64 type

df_weather = df_convert_dtypes(df_weather, np.int64, np.float64)
```

[21]:
```
# Convert dt_iso to datetime type, rename it and set it as index

df_weather['time'] = pd.to_datetime(df_weather['dt_iso'], utc=True, infer_datetime_format=True)
df_weather = df_weather.drop(['dt_iso'], axis=1)
df_weather = df_weather.set_index('time')
```

We have to **split** the `df_weather` dataset into 5 datasets, **one for each different city** (Madrid, Barcelona, Bilbao, Seville and Valencia). But first, let's see the average values for each column, grouped by each city (note that the 'weather_id' average has no meaning whatsoever).

[22]:
```
# Display average weather features grouped by each city

mean_weather_by_city = df_weather.groupby('city_name').mean()
mean_weather_by_city
```

[22]:
```
                  temp    temp_min    temp_max     pressure   humidity  \
city_name
 Barcelona  289.848248  288.594704  291.021987  1284.010486  73.994221
Bilbao      286.378489  284.916661  288.036687  1017.567439  79.089455
Madrid      288.061071  286.824877  289.155600  1011.838448  59.776932
Seville     293.105431  291.184103  295.962431  1018.504711  64.140732
Valencia    290.780780  290.222277  291.355025  1015.973794  65.145113

            wind_speed    wind_deg   rain_1h   rain_3h   snow_3h  clouds_all  \
city_name
 Barcelona    2.786588  187.188043  0.117079  0.000327  0.000000   23.229648
Bilbao        1.957470  159.883536  0.123493  0.001034  0.023455   43.960697
Madrid        2.441696  173.293159  0.055083  0.000129  0.000029   22.397028
Seville       2.483787  151.757179  0.045392  0.000180  0.000000   14.748770
Valencia      2.692815  160.753820  0.035924  0.000226  0.000154   20.820999

            weather_id
city_name
 Barcelona  760.917465
Bilbao      723.943228
Madrid      762.260264
Seville     771.409849
Valencia    781.228283
```

[23]:
```
# Find NaNs and duplicates in df_weather

print('There are {} missing values or NaNs in df_weather.'
      .format(df_weather.isnull().values.sum()))

temp_weather = df_weather.duplicated(keep='first').sum()

print('There are {} duplicate rows in df_weather based on all columns.'
```

```
      .format(temp_weather))
```

```
There are 0 missing values or NaNs in df_weather.
There are 8622 duplicate rows in df_weather based on all columns.
```

It seems that `df_weather` has a lot of *duplicate values*. However, **the method above may also show us rows which have the exame same values**. This is not what we are looking for. What we want to ensure, is that **there are no duplicate index rows**, i.e. that we do not have multiple rows **for the exact same hour**. Of course, we also have to make sure that these duplicates concern **each individual city**. Since, `df_weather` contains information about 5 different cities, it is very useful to display the number of observations for each one and compare it with the size of `df_energy`.

[24]:
```
# Display the number of rows in each dataframe

print('There are {} observations in df_energy.'.format(df_energy.shape[0]))

cities = df_weather['city_name'].unique()
grouped_weather = df_weather.groupby('city_name')

for city in cities:
    print('There are {} observations in df_weather'
          .format(grouped_weather.get_group('{}'.format(city)).shape[0]),
          'about city: {}.'.format(city))
```

```
There are 35064 observations in df_energy.
There are 35145 observations in df_weather about city: Valencia.
There are 36267 observations in df_weather about city: Madrid.
There are 35951 observations in df_weather about city: Bilbao.
There are 35476 observations in df_weather about city:  Barcelona.
There are 35557 observations in df_weather about city: Seville.
```

As we can see, the two dataframes (`df_energy` and `df_weather`) cannot be merged yet. There are many duplicates for every city in `df_weather` and we should drop them and see if their number of rows match. We do this by resetting the index, keeping only the **first rows** which have the same 'time' and 'city_name' values and then setting again 'time' as the index. For further research on the dataset, let us also create a second dataframe, `df_weather_2` in which we do the same procedure, but keep only the **last rows** which have the same 'time' and 'city_name'.

[25]:
```
# Create df_weather_2 and drop duplicate rows in df_weather

df_weather_2 = df_weather.reset_index().drop_duplicates(subset=['time', 'city_name'],
                                                        keep='last').set_index('time')

df_weather = df_weather.reset_index().drop_duplicates(subset=['time', 'city_name'],
                                                      keep='first').set_index('time')
```

[26]:
```
# Display the number of rows in each dataframe again

print('There are {} observations in df_energy.'.format(df_energy.shape[0]))

grouped_weather = df_weather.groupby('city_name')

for city in cities:
    print('There are {} observations in df_weather'
          .format(grouped_weather.get_group('{}'.format(city)).shape[0]),
          'about city: {}.'.format(city))
```

```
There are 35064 observations in df_energy.
There are 35064 observations in df_weather about city: Valencia.
There are 35064 observations in df_weather about city: Madrid.
There are 35064 observations in df_weather about city: Bilbao.
There are 35064 observations in df_weather about city:  Barcelona.
There are 35064 observations in df_weather about city: Seville.
```

The column 'weather_icon' is irrelevant for our analysis, so we will drop it. Furthermore, the columns 'weather_main' and 'weather_description' contain approximately the same information as the column 'weather_id'; the information concerns a qualitative description of the weather at the given hour. So, we will work with only one of them. However, in order to make a choice, we have to check the **unique values** as well as the **consistency** of each column.

[27]:
```
# Display all the unique values in the column 'weather_description'

weather_description_unique = df_weather['weather_description'].unique()
weather_description_unique
```

[27]:
```
array(['sky is clear', 'few clouds', 'scattered clouds', 'broken clouds',
       'overcast clouds', 'light rain', 'moderate rain',
       'heavy intensity rain', 'mist', 'heavy intensity shower rain',
       'shower rain', 'very heavy rain', 'thunderstorm with heavy rain',
       'thunderstorm with light rain', 'proximity thunderstorm',
       'thunderstorm', 'light intensity shower rain',
       'light intensity drizzle', 'thunderstorm with rain', 'fog',
       'smoke', 'drizzle', 'heavy intensity drizzle', 'haze',
       'proximity shower rain', 'light snow', 'rain and snow',
       'light rain and snow', 'snow', 'sleet', 'rain and drizzle',
       'light intensity drizzle rain', 'light shower snow',
       'proximity moderate rain', 'ragged shower rain', 'heavy snow',
       'sand dust whirls', 'proximity drizzle', 'dust',
       'light thunderstorm', 'squalls'], dtype=object)
```

[28]:
```
# Display all the unique values in the column 'weather_main'

weather_main_unique = df_weather['weather_main'].unique()
weather_main_unique
```

[28]:
```
array(['clear', 'clouds', 'rain', 'mist', 'thunderstorm', 'drizzle',
       'fog', 'smoke', 'haze', 'snow', 'dust', 'squall'], dtype=object)
```

[29]:
```
# Display all the unique values in the column 'weather_id'

weather_id_unique = df_weather['weather_id'].unique()
weather_id_unique
```

[29]:
```
array([800., 801., 802., 803., 804., 500., 501., 502., 701., 522., 521.,
       503., 202., 200., 211., 520., 300., 201., 741., 711., 301., 302.,
       721., 600., 616., 615., 601., 611., 311., 310., 620., 531., 602.,
       731., 761., 210., 771.])
```

We can see that in terms of qualitative description, the 'weather_main' column seems to contain the less detailed -or "poorest"- information, while 'weather_id' and 'weather_description' have more complex information and approximately the same number of unique values.

Nevertheless, it is also useful to check the consistency of the information in each column. Since our dataset contained duplicate rows and we utilized two different methods for cleaning it, a very good way to check the consistency of the data in these three columns would be to compare the two cleaned dataframes, `df_weather` and `df_weather_2`. In order to do this, we employ the R^2 ("R-squared" or "coefficient of determination") metric, after encoding 'weather_description' and 'weather_main' from strings to numerical labels.

[30]:
```
# Define a function which will calculate R-squared score for the same column in two dataframes

def encode_and_display_r2_score(df_1, df_2, column, categorical=False):
    dfs = [df_1, df_2]
    if categorical:
        for df in dfs:
            le = LabelEncoder()
            df[column] = le.fit_transform(df[column])
    r2 = r2_score(df_1[column], df_2[column])
```

```
    print("R-Squared score of {} is {}".format(column, r2.round(3)))
```

[31]:
```
# Display the R-squared scores for the columns with qualitative weather descriptions in df_weather and
↪df_weather_2

encode_and_display_r2_score(df_weather, df_weather_2, 'weather_description', categorical=True)
encode_and_display_r2_score(df_weather, df_weather_2, 'weather_main', categorical=True)
encode_and_display_r2_score(df_weather, df_weather_2, 'weather_id')
```

```
R-Squared score of weather_description is 0.973
R-Squared score of weather_main is 0.963
R-Squared score of weather_id is 0.921
```

It seems that we have found incosistencies in our dataset. In particular, all three of the columns which contain qualitative info for the condition of the weather seem to contain a large part of the duplicates that we found earlier in `df_weather`. From the above R-squared scores, it seems that keeping and one-hot encoding 'weather_description' or 'weather_main' is the best idea to get the most out of information in the dataset. Note that **the R^2 values of these two are not comparable** because we have first encoded them; in order to actually calculate the similarity between two strings, we need to employ a different metric such as the *Levenshtein distance*, i.e. the number of edits required to transform string A to string B. In this case, even this metric would not be enough in order to make 'weather_main' and 'weather_description' comparable as the former contains only single words while the latter large phrases which describe the weather.

[32]:
```
# Drop columns with qualitative weather information
df_weather = df_weather.drop(['weather_main', 'weather_id',
                              'weather_description', 'weather_icon'], axis=1)
```

We have to emphasize that the method we employed above in order to check the consistency of our dataset **is not 100% accurate**. This is because it compares two dataframes from which we have cleaned the duplicates in **only two ways exclusively**; keeping the first rows and keeping the last rows of the duplicates. Nevertheless, it gives good results and can be used to check the values of the other columns as well. We should not that since we removed the duplicates based on all the columns except 'time' and 'city_name', there is no point in checking the latter.

[33]:
```
# Display the R-squared for all the columns in df_weather and df_weather_2

df_weather_cols = df_weather.columns.drop('city_name')

for col in df_weather_cols:
    encode_and_display_r2_score(df_weather, df_weather_2, col)
```

```
R-Squared score of temp is 1.0
R-Squared score of temp_min is 1.0
R-Squared score of temp_max is 1.0
R-Squared score of pressure is 1.0
R-Squared score of humidity is 1.0
R-Squared score of wind_speed is 1.0
R-Squared score of wind_deg is 1.0
R-Squared score of rain_1h is 1.0
R-Squared score of rain_3h is 1.0
R-Squared score of snow_3h is 1.0
R-Squared score of clouds_all is 1.0
```

The above confirms our intuition that 'weather_description', 'weather_main' and 'weather_id' are the sole culprits for the duplicates in `df_weather` and although we are not absolutely certain for this (for the reasons we described above), it is more than enough to reaffirm the validity and consistency of the rest of the columns in `df_weather` which contain numerical values.

[34]:
```
# Display the number of duplicates in df_weather

temp_weather = df_weather.reset_index().duplicated(subset=['time', 'city_name'],
                                                   keep='first').sum()
```

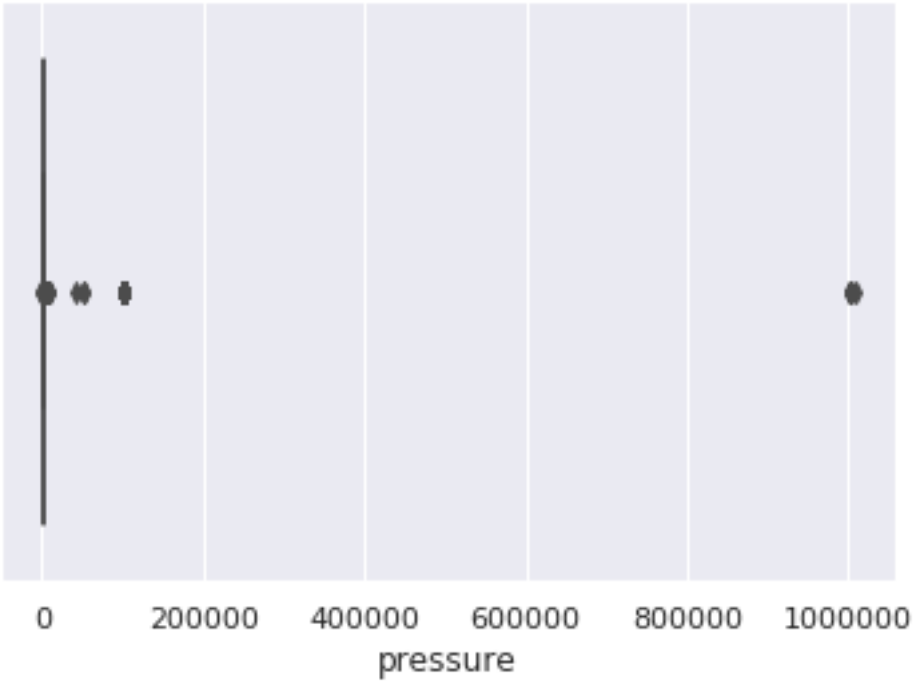
```
print('There are {} duplicate rows in df_weather ' \
      'based on all columns except "time" and "city_name".'.format(temp_weather))
```

```
There are 0 duplicate rows in df_weather based on all columns except "time" and
"city_name".
```

Before proceeding to the next step, i.e. merging `df_energy` and `df_weather`, we also want to treat the outliers which we found earlier in 'pressure' and 'wind_speed'. We will visualize the outliers in these columns using boxplot, change their values to NaNs and then use a linear interpolation in order to replace their values.

[35]:
```
# Check for outliers in 'pressure' column

sns.boxplot(x=df_weather['pressure'])
plt.show()
```



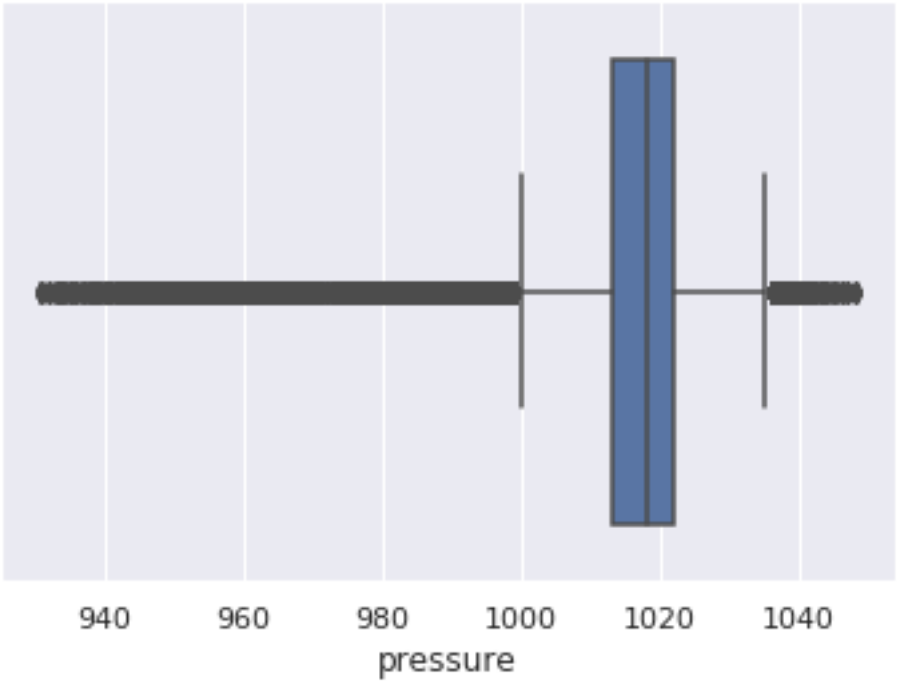
Even a pressure of approximately 100,000 HPa or 10 MPa, which is clearly visible in the above figure, corresponds to a quantity greater than the atmospheric pressure of Venus [2]. In order to be sure, we will set as **NaN** every value in the 'pressure' column which is **higher than 1051 hPa**, which is just above the highest air pressure ever recorded in the Iberian peninsula [6]. While outliers on the low side are not visible in the boxplot above, it is a good idea to also replace the values which are **lower than 931 hPa**, i.e. the lowest air pressure ever recorded in the Iberian peninsula [6].

[36]:
```
# Replace outliers in 'pressure' with NaNs

df_weather.loc[df_weather.pressure > 1051, 'pressure'] = np.nan
df_weather.loc[df_weather.pressure < 931, 'pressure'] = np.nan
```

[37]:
```
# Check for outliers in 'pressure' column again

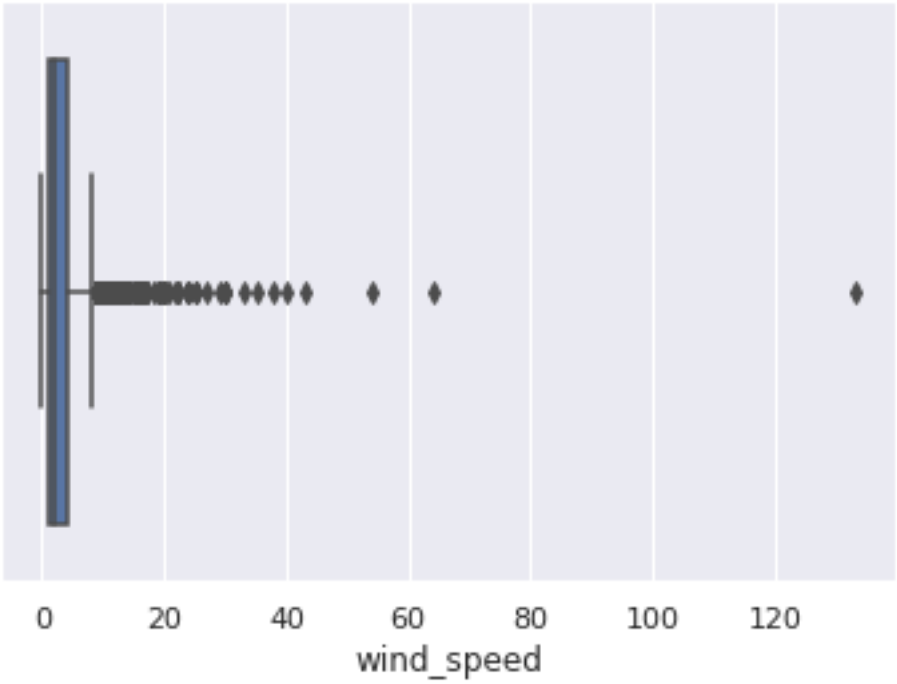
sns.boxplot(x=df_weather['pressure'])
plt.show()
```



Things look a lot better regarding atmospheric pressure ('pressure') as the column has been cleaned of extreme-case outliers. What about 'wind_speed' however?

[38]:
```
# Check for outliers in 'wind_speed' column

sns.boxplot(x=df_weather['wind_speed'])
plt.show()
```



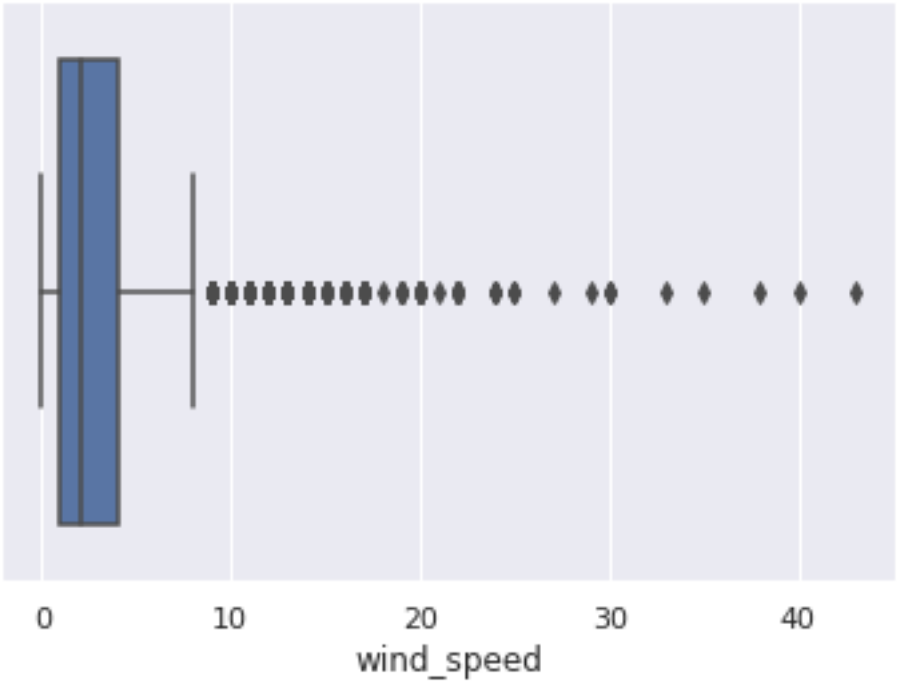
Here, we will follow a conservative approach as above. We will set as **NaNs** the values in 'wind_speed' which are **higher than 50 m/s**, which is the highest bound (112 mph) of the wind speed estimate of a F1 (Fujita scale) tornado [4], having in mind that the last F1 tornado in Spain was Gandia tornado, which was recorded on 28th September 2012 [5], i.e. roughly 2 years before the starting point of our available data.

[39]:
```
# Replace outliers in 'wind_speed' with NaNs

df_weather.loc[df_weather.wind_speed > 50, 'wind_speed'] = np.nan
```

[40]:
```
# Check for outliers in 'wind_speed' column again

sns.boxplot(x=df_weather['wind_speed'])
plt.show()
```



[41]:
```
# Fill null values using interpolation

df_weather.interpolate(method='linear', limit_direction='forward', inplace=True, axis=0)
```

Everything looks nice and clean and `df_weather` is also ready for use as input to our model. We also have to plot the 'rain_1h' and 'rain_3h' columns, as we have indicated that at least one of them (probably 'rain_3h') is quite problematic. Nevertheless, we will do so after we merge the two datasets.

3.3 1.3. Merging the two datasets

[42]:
```
# Split the df_weather into 5 dataframes (one for each city)

df_1, df_2, df_3, df_4, df_5 = [x for _, x in df_weather.groupby('city_name')]
dfs = [df_1, df_2, df_3, df_4, df_5]
```

[43]:
```
# Merge all dataframes into the final dataframe

df_final = df_energy

for df in dfs:
    city = df['city_name'].unique()
    city_str = str(city).replace("'", "").replace('[', '').replace(']', '').replace(' ', '')
    df = df.add_suffix('_{}'.format(city_str))
    df_final = df_final.merge(df, on=['time'], how='outer')
    df_final = df_final.drop('city_name_{}'.format(city_str), axis=1)

df_final.columns
```

[43]:
```
Index(['generation biomass', 'generation fossil brown coal/lignite',
       'generation fossil gas', 'generation fossil hard coal',
       'generation fossil oil', 'generation hydro pumped storage consumption',
       'generation hydro run-of-river and poundage',
```

```
       'generation hydro water reservoir', 'generation nuclear',
       'generation other', 'generation other renewable', 'generation solar',
       'generation waste', 'generation wind onshore', 'total load actual',
       'price day ahead', 'price actual', 'temp_Barcelona',
       'temp_min_Barcelona', 'temp_max_Barcelona', 'pressure_Barcelona',
       'humidity_Barcelona', 'wind_speed_Barcelona', 'wind_deg_Barcelona',
       'rain_1h_Barcelona', 'rain_3h_Barcelona', 'snow_3h_Barcelona',
       'clouds_all_Barcelona', 'temp_Bilbao', 'temp_min_Bilbao',
       'temp_max_Bilbao', 'pressure_Bilbao', 'humidity_Bilbao',
       'wind_speed_Bilbao', 'wind_deg_Bilbao', 'rain_1h_Bilbao',
       'rain_3h_Bilbao', 'snow_3h_Bilbao', 'clouds_all_Bilbao', 'temp_Madrid',
       'temp_min_Madrid', 'temp_max_Madrid', 'pressure_Madrid',
       'humidity_Madrid', 'wind_speed_Madrid', 'wind_deg_Madrid',
       'rain_1h_Madrid', 'rain_3h_Madrid', 'snow_3h_Madrid',
       'clouds_all_Madrid', 'temp_Seville', 'temp_min_Seville',
       'temp_max_Seville', 'pressure_Seville', 'humidity_Seville',
       'wind_speed_Seville', 'wind_deg_Seville', 'rain_1h_Seville',
       'rain_3h_Seville', 'snow_3h_Seville', 'clouds_all_Seville',
       'temp_Valencia', 'temp_min_Valencia', 'temp_max_Valencia',
       'pressure_Valencia', 'humidity_Valencia', 'wind_speed_Valencia',
       'wind_deg_Valencia', 'rain_1h_Valencia', 'rain_3h_Valencia',
       'snow_3h_Valencia', 'clouds_all_Valencia'],
      dtype='object')
```

[44]:
```
# Display the number of NaNs and duplicates in the final dataframe

print('There are {} missing values or NaNs in df_final.'
      .format(df_final.isnull().values.sum()))

temp_final = df_final.duplicated(keep='first').sum()

print('\nThere are {} duplicate rows in df_energy based on all columns.'
      .format(temp_final))
```

```
There are 0 missing values or NaNs in df_final.

There are 0 duplicate rows in df_energy based on all columns.
```

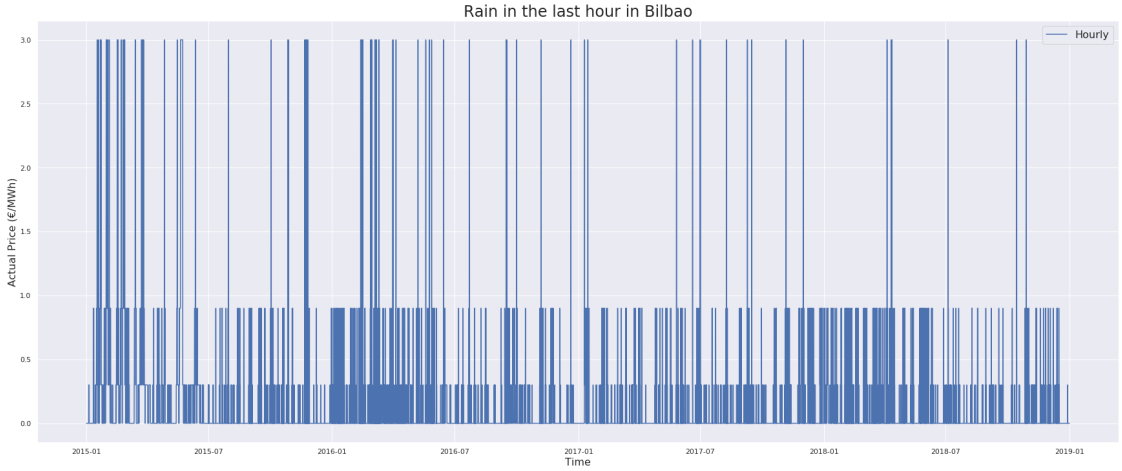
4 2. Visualizations and Time Series Analysis

4.1 2.1. Useful visualizations and insights

Before anything else, we will check the 'rain_1h' and 'rain_3h' columns for a specific city.

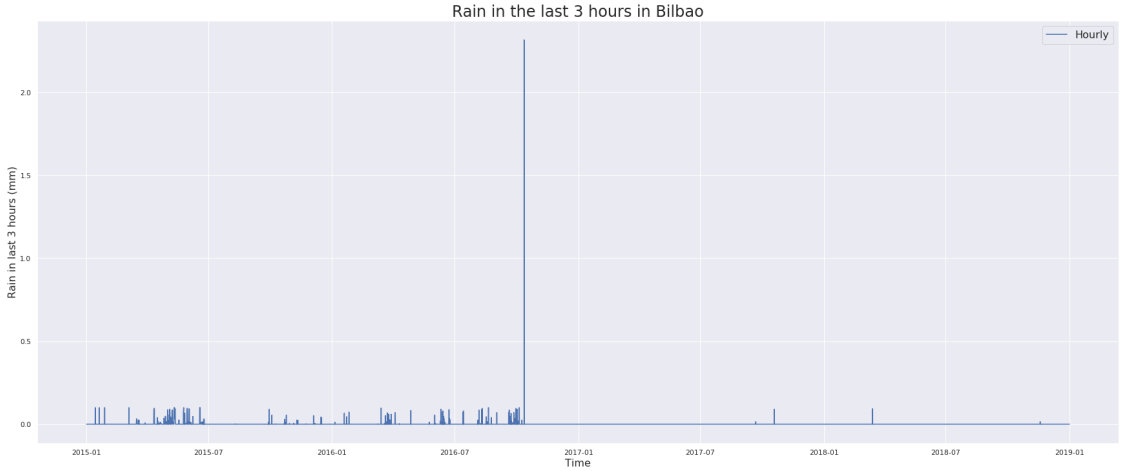
[45]:
```
# Plot the 'rain_1h' for Bilbao

ax = plot_series(df_final, 'rain_1h_Bilbao',
                 label='Hourly', ylabel='Actual Price (€/MWh)',
                 title='Rain in the last hour in Bilbao')
plt.show()
```



[46]:
```
# Plot the 'rain_3h' for Bilbao

ax = plot_series(df_final, 'rain_3h_Bilbao',
                 label='Hourly', ylabel='Rain in last 3 hours (mm)',
                 title='Rain in the last 3 hours in Bilbao')
plt.show()
```



From the two figures aboce, we can conclude that **'rain_3h' is an unreliable feature which will drop from the dataset**.

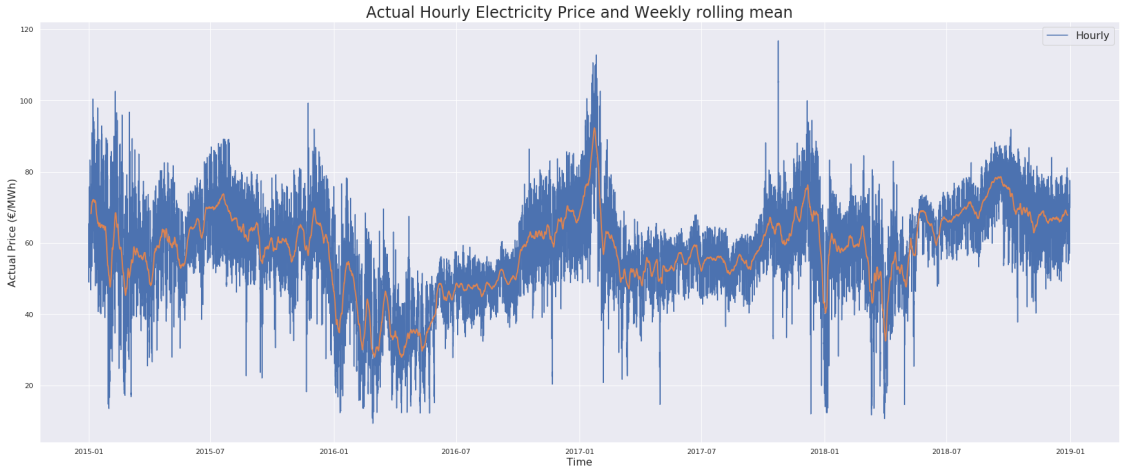
[47]:
```
cities = ['Barcelona', 'Bilbao', 'Madrid', 'Seville', 'Valencia']

for city in cities:
    df_final = df_final.drop(['rain_3h_{}'.format(city)], axis=1)
```

[48]:
```
# Plot the hourly actual electricity price, along with the weekly rolling mean

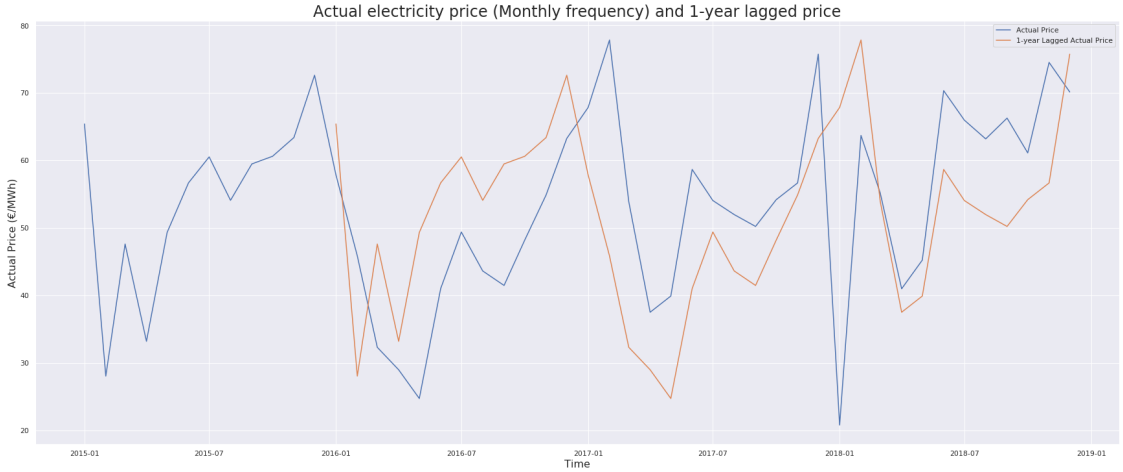
rolling = df_final['price actual'].rolling(24*7, center=True).mean()
ax = plot_series(df_final, 'price actual', label='Hourly', ylabel='Actual Price (€/MWh)',
                 title='Actual Hourly Electricity Price and Weekly rolling mean')
ax.plot(rolling, linestyle='-', linewidth=2, label='Weekly rolling mean')
```

```
plt.show()
```



[49]:
```
# Plot the electricity price (monthly frequence) along with its 1-year lagged series

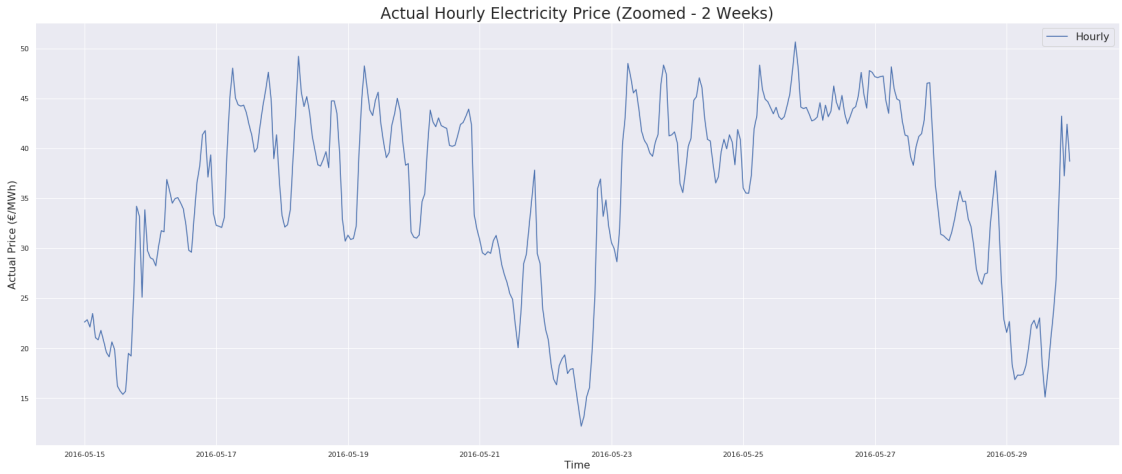
monthly_price = df_final['price actual'].asfreq('M')
ax = plot_series(series=monthly_price, ylabel='Actual Price (€/MWh)',
                 title='Actual electricity price (Monthly frequency) and 1-year lagged price')
shifted = df_final['price actual'].asfreq('M').shift(12)
ax.plot(shifted, label='Hourly')
ax.legend(['Actual Price', '1-year Lagged Actual Price'])
plt.show()
```



The figure above shows the (resampled) monthly frequency of the actual electricity price, along with its 1-year lagged monthly frequency. From this, we can see that there are indeed **seasonal patterns at the monthly scale**, as certain "spikes" in the time-series take place in exactly the same months. This means that it would be a good idea to create a new feature for the months. If the figure displayed the 1-year lagged actual energy price at hourly or weekly time scales, we would -of course- see no such seasonal patterns.

[50]:
```
# Plot the actual electricity price at a daily/weekly scale

ax = plot_series(df_final, 'price actual', label='Hourly', ylabel='Actual Price (€/MWh)',
                 start=1 + 24 * 500, end=1 + 24 * 515,
                 title='Actual Hourly Electricity Price (Zoomed - 2 Weeks)')
plt.show()
```

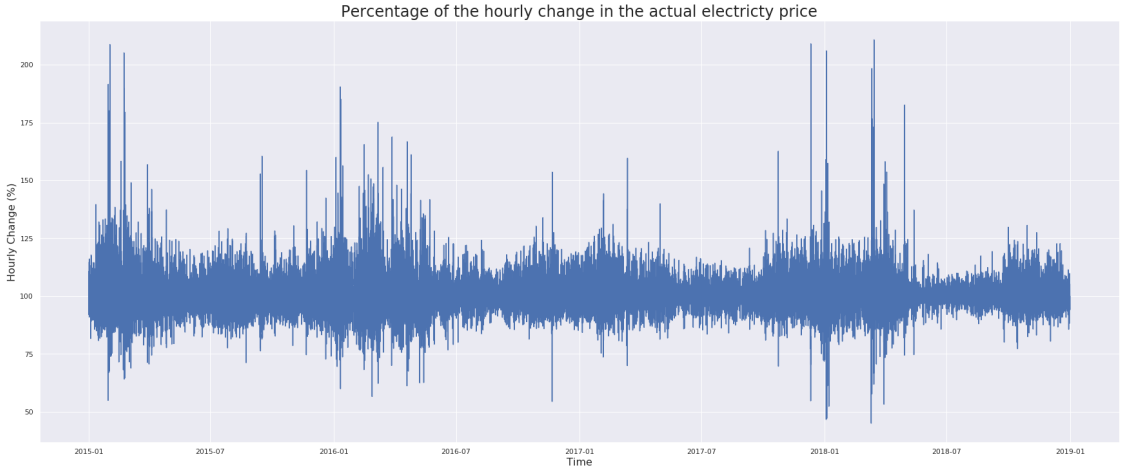


In the figure above, we have plotted that actual hourly electricity price from 15/06/2016 (Sunday) at 00:00 up to 29/06/2016 (Sunday) at 23:00, i.e. two weeks of data. We can observe that there are many patterns and periodicities, such as: - A periodicity from week to week, as the electricity price tends to be higher during business days and lower during weekends and especially during sundays. - An intradyay periodicity, as the price is higher during the day and lower during the night. - A periodicity within the business hours, as in some cases the electricity price drops for a few hours, which is probably due to "siesta", the traditional lunch break between 01:30PM and 04:30PM, as Spain does not strictly follow the 9AM-5PM business day.

Later on, we will make sure that we will generate features which contain these kinds of information.

[51]:
```
# Plot the percentage of the hourly change in the actual electricity price

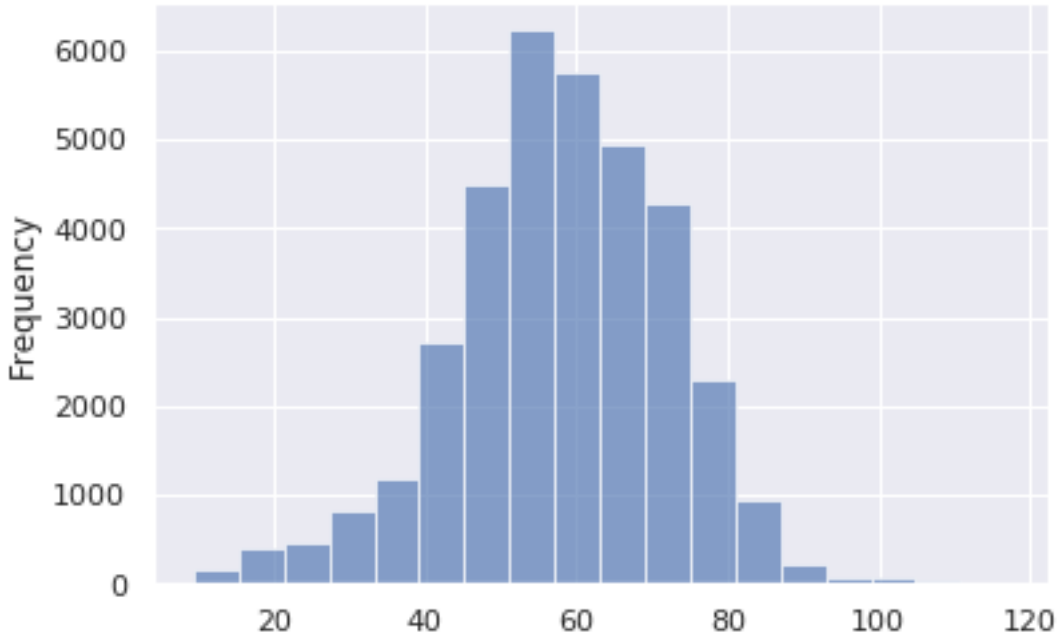
change = df_energy['price actual'].div(df_energy['price actual'].shift(1)).mul(100)
ax = plot_series(series=change, ylabel='Hourly Change (%)',
                 title='Percentage of the hourly change in the actual electricty price')
plt.show()
```



From the figure above, we can see that the change in the actual price from hour to hour is in most of the cases between -25% (actual price * 0.75) and +25% (actual price * 1.25). However, there are also very few outliers which show that the price gets halved (-50%) or doubled (+100%).

[52]:
```
# Plot the histogram of the actual electricity price

ax = df_energy['price actual'].plot.hist(bins=18, alpha=0.65)
```



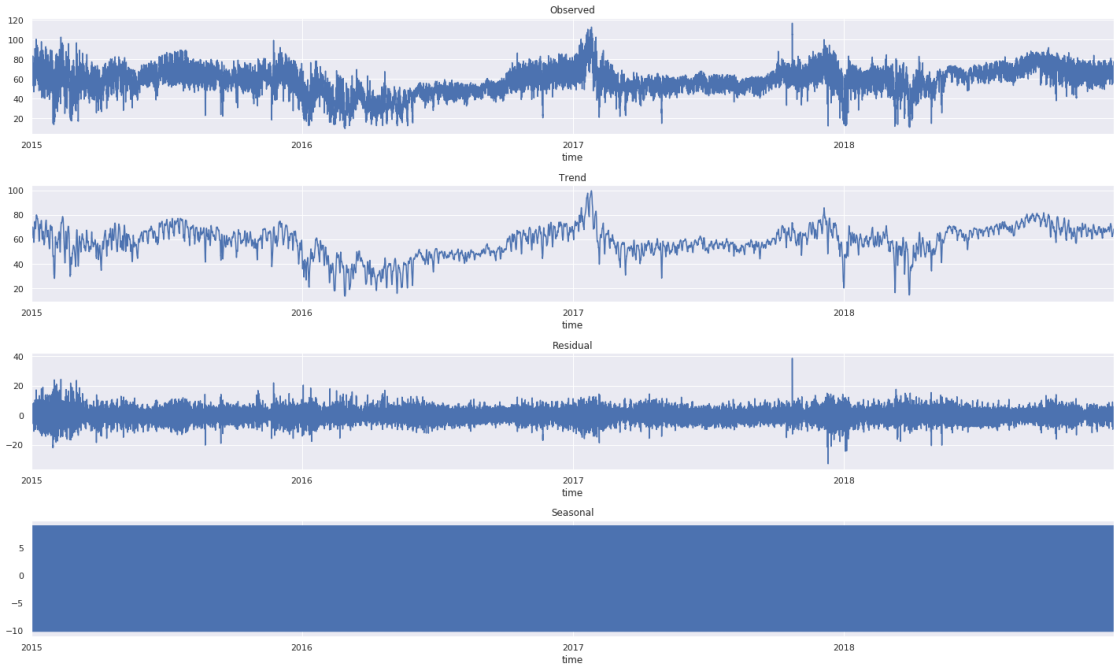
In the figure above, we can see that the actual energy price roughly follows a normal distribution and thus, could be standardized. However, we also have to make sure that the time series does not require any other kinds of transformations. More specifically, we will check whether the time series of energy price is stationary, after visualizing its decomposed component time-series.

4.2 2.2. Decomposition and stationarity tests

[53]:
```
# Decompose the electricity price time series

res = sm.tsa.seasonal_decompose(df_energy['price actual'], model='additive')

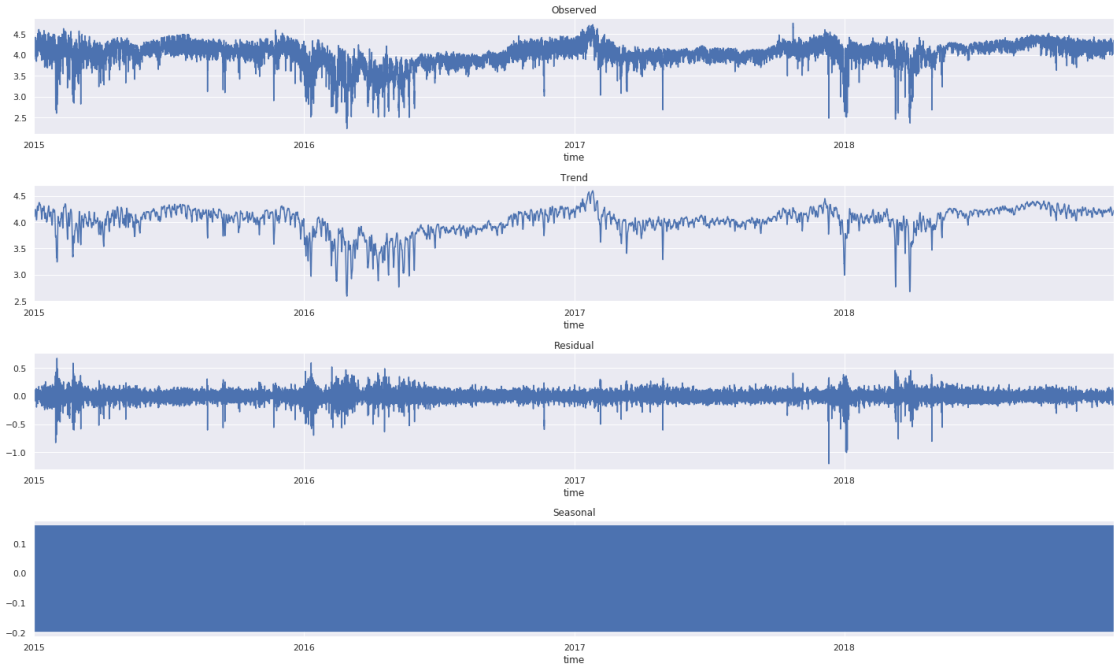
fig, (ax1, ax2, ax3, ax4) = plt.subplots(4, 1, figsize=(20, 12))
res.observed.plot(ax=ax1, title='Observed')
res.trend.plot(ax=ax2, title='Trend')
res.resid.plot(ax=ax3, title='Residual')
res.seasonal.plot(ax=ax4, title='Seasonal')
plt.tight_layout()
plt.show()
```



[54]:
```
# Decompose the log electricity price time-series

res = sm.tsa.seasonal_decompose(np.log(df_energy['price actual']), model='additive')

fig, (ax1, ax2, ax3, ax4) = plt.subplots(4, 1, figsize=(20, 12))
res.observed.plot(ax=ax1, title='Observed')
res.trend.plot(ax=ax2, title='Trend')
res.resid.plot(ax=ax3, title='Residual')
res.seasonal.plot(ax=ax4, title='Seasonal')
plt.tight_layout()
plt.show()
```



The **Augmented Dickey-Fuller (ADF) test**, a type of unit root test, determines how strongly a time series is defined by a trend. Its hypotheses are the following: - **Null Hypothesis** H_0: There is a unit root in the time series, i.e. the series is autocorrelated with (r=1), a time dependent structure and thus, is not stationary. - **Alternate Hypothesis** H_1: The time series has no unit root and is either stationary or can be made stationary using differencing.

[55]:
```
y = df_final['price actual']
adf_test = adfuller(y, regression='c')
print('ADF Statistic: {:.6f}\np-value: {:.6f}\n#Lags used: {}'
      .format(adf_test[0], adf_test[1], adf_test[2]))
for key, value in adf_test[4].items():
    print('Critical Value ({}): {:.6f}'.format(key, value))
```

```
ADF Statistic: -9.147016
p-value: 0.000000
#Lags used: 50
Critical Value (1%): -3.430537
Critical Value (5%): -2.861623
Critical Value (10%): -2.566814
```

The ADF statistic (-9.147) is less than the critical value at 1% (-3.431) and thus, we can say that **we reject the null hypothesis H_0 with a significance level 1%**, meaning that there is not a root-unit in the time series and thus, that it is either stationary or could be made stationary with 1st order differencing (difference-stationary).

The **Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test**, follows the opposite logic from Augmented Dickey-Fuller test and checks for stationarity. Its hypotheses are the following: - **Null Hypothesis** H_0: The time series is level, i.e. it is stationary around a constant. - **Alternate Hypothesis** H_1: There is a unit root in the time series and thus it not stationary.

[56]:
```
kpss_test = kpss(y, regression='c', lags='legacy')
print('KPSS Statistic: {:.6f}\np-value: {:.6f}\n#Lags used: {}'
      .format(kpss_test[0], kpss_test[1], kpss_test[2]))
for key, value in kpss_test[3].items():
```

```
    print('Critical Value ({}): {:.6f}'.format(key, value))
```

```
KPSS Statistic: 7.957007
p-value: 0.010000
#Lags used: 52
Critical Value (10%): 0.347000
Critical Value (5%): 0.463000
Critical Value (2.5%): 0.574000
Critical Value (1%): 0.739000
```

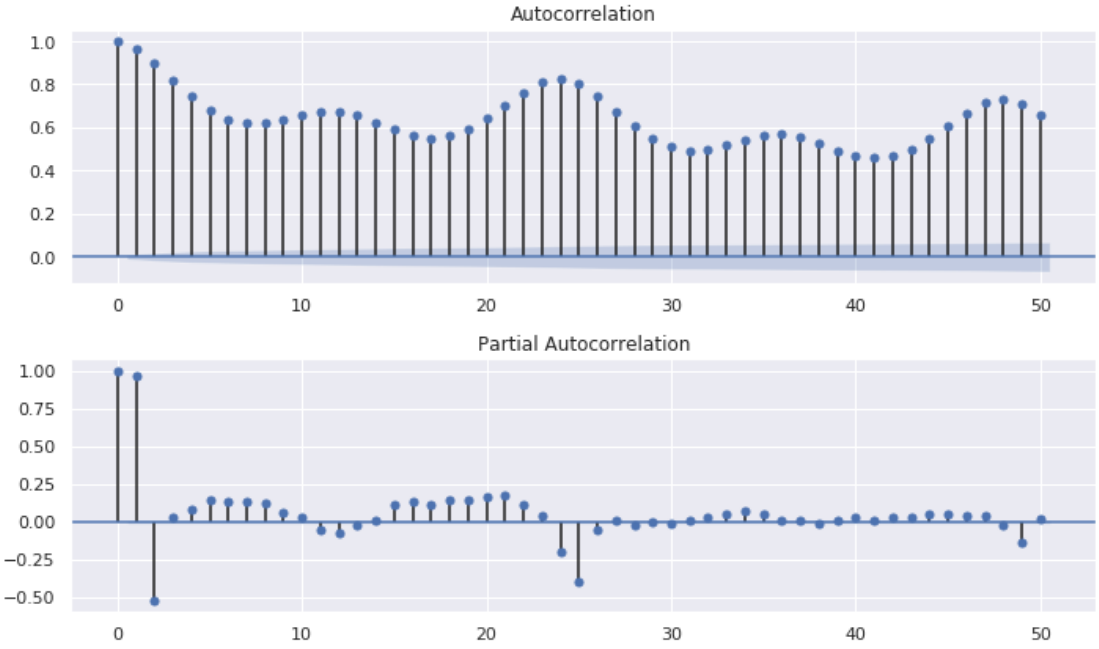
The KPSS statistic (7.957) is higher than the critical value at 1% (0.739) and thus, we can say that **we cannot reject the null hypothesis H_0 with a significance level 1%**, meaning that the time series is stationary or stationary around a constant.

Both tests concluded that the electricity price time series is stationary, while it is also true that deep neural networks can handle such properties in a more forgiving way compared to ARIMA models.

4.3 2.3. Autocorrelation, partial autocorrelation and cross-correlation

[57]:
```
# Plot autocorrelation and partial autocorrelation plots

fig, (ax1, ax2) = plt.subplots(nrows=2, figsize=(10, 6))
plot_acf(df_final['price actual'], lags=50, ax=ax1)
plot_pacf(df_final['price actual'], lags=50, ax=ax2)
plt.tight_layout()
plt.show()
```

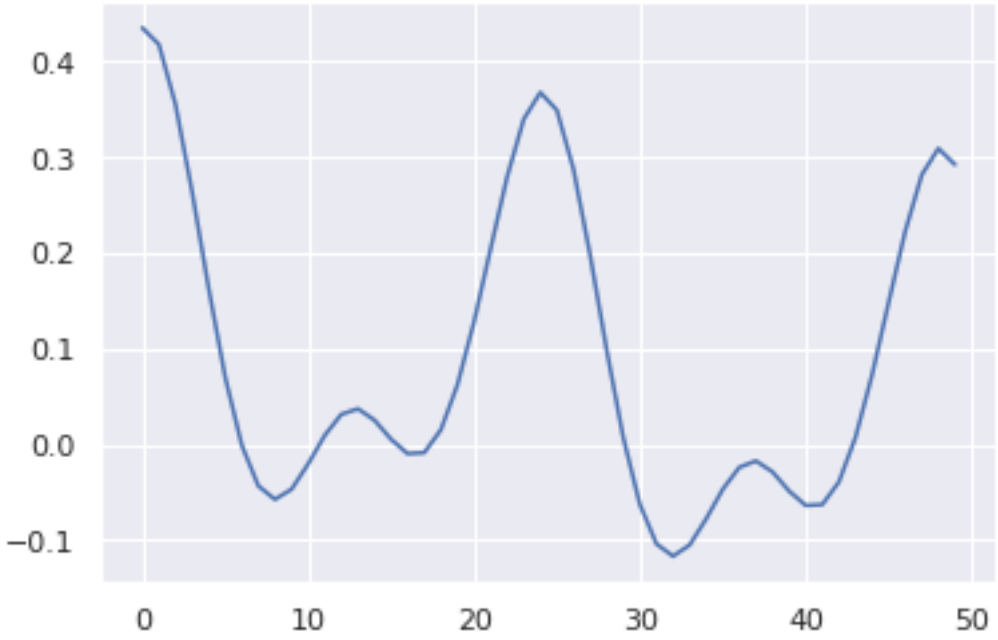


The partial autocorrelation plot of the eletricity price time series shows that the direct relationship between an observation at a given hour (t) is strongest with the observations at t-1, t-2, t-24 and t-25 time-steps and diminishes afterwards. Thus, we are going to use the 25 previous values of each time series which will constitute a feature for our models.

Nevertheless, it would quite definitely be more beneficial if we only chose to use specific past values (observations at certain time-lags) of a given feature, based on the cross-correlation between the electricity price

and each one of the features in the dataset. For example, below we can see the cross-correlation between the electricity price and the total load. We see that there are many time-lags with a correlation which is close to zero and could be ommited.

[58]:
```
cross_corr = ccf(df_final['total load actual'], df_final['price actual'])
plt.plot(cross_corr[0:50])
plt.show()
```



For simplicity, we will just work with the correlations between the electricity price and the other features at each given hour.

[59]:
```
# Find the correlations between the electricity price and the rest of the features

correlations = df_final.corr(method='pearson')
print(correlations['price actual'].sort_values(ascending=False).to_string())
```

```
price actual                                 1.000000
price day ahead                              0.732155
generation fossil hard coal                  0.465637
generation fossil gas                        0.461452
total load actual                            0.435253
generation fossil brown coal/lignite         0.363993
generation fossil oil                        0.285050
generation other renewable                   0.255551
pressure_Barcelona                           0.249177
pressure_Bilbao                              0.194063
generation waste                             0.168710
generation biomass                           0.142671
temp_min_Valencia                            0.133141
pressure_Valencia                            0.109812
temp_min_Barcelona                           0.103726
generation other                             0.099914
generation solar                             0.098529
temp_max_Madrid                              0.096279
temp_Valencia                                0.090505
pressure_Seville                             0.090162
```

```
temp_Madrid                                  0.087995
temp_Barcelona                               0.085857
humidity_Valencia                            0.078819
temp_min_Seville                             0.077296
temp_max_Bilbao                              0.076766
temp_min_Bilbao                              0.074776
temp_Bilbao                                  0.073018
generation hydro water reservoir             0.071910
temp_max_Barcelona                           0.068936
temp_min_Madrid                              0.066777
temp_Seville                                 0.050274
temp_max_Valencia                            0.047478
clouds_all_Valencia                          0.040055
pressure_Madrid                              0.018756
snow_3h_Bilbao                               0.014920
rain_1h_Valencia                             0.012049
snow_3h_Valencia                             0.007461
temp_max_Seville                             0.003253
humidity_Bilbao                             -0.000450
snow_3h_Madrid                              -0.008427
rain_1h_Madrid                              -0.027137
clouds_all_Barcelona                        -0.027599
rain_1h_Seville                             -0.034887
humidity_Barcelona                          -0.037682
generation nuclear                          -0.053016
rain_1h_Barcelona                           -0.055130
humidity_Madrid                             -0.064668
wind_speed_Seville                          -0.078469
rain_1h_Bilbao                              -0.078806
clouds_all_Madrid                           -0.079415
wind_deg_Madrid                             -0.082756
clouds_all_Seville                          -0.086233
wind_deg_Valencia                           -0.092710
wind_deg_Barcelona                          -0.096248
humidity_Seville                            -0.103004
wind_deg_Bilbao                             -0.103097
clouds_all_Bilbao                           -0.132669
generation hydro run-of-river and poundage  -0.136659
wind_deg_Seville                            -0.137099
wind_speed_Barcelona                        -0.138658
wind_speed_Valencia                         -0.142360
wind_speed_Bilbao                           -0.143327
generation wind onshore                     -0.220497
wind_speed_Madrid                           -0.245861
generation hydro pumped storage consumption -0.426196
snow_3h_Barcelona                                 NaN
snow_3h_Seville                                   NaN
```

From the above, we can already see some very interesting correlations among the energy price that we want to predict and the rest of the features. For example, the total energy load and the amount of energy generated from sources related to fossil fuels, is positively correlated with the electricity price. In contrast, the wind speed in almost every city and the amount of storage energy cosumed through hydroelectric pumping is negatively correlated with the energy price. We will go ahead and drop the 'snow_3h_Barcelona' and 'snow_3h_Seville' which give NaNs in their correlations with the electricity actual price.

[60]:
```
df_final = df_final.drop(['snow_3h_Barcelona', 'snow_3h_Seville'], axis=1)
```

[61]:
```
# Plot Pearson correlation matrix

correlations = df_final.corr(method='pearson')
fig = plt.figure(figsize=(24, 24))
sns.heatmap(correlations, annot=True, fmt='.2f')
plt.title('Pearson Correlation Matrix')
plt.show()
```



While we cannot make a lot out of the above correlation matrix, we can observe that there quite a lot of features that are highly correlated with each other.

[62]:
```
highly_correlated = abs(correlations[correlations > 0.75])
print(highly_correlated[highly_correlated < 1.0].stack().to_string())
```

```
generation fossil brown coal/lignite  generation fossil hard coal
0.768831
generation fossil hard coal           generation fossil brown coal/lignite
0.768831
temp_Barcelona                        temp_min_Barcelona
0.970264

                                      temp_max_Barcelona
0.976904

                                      temp_Bilbao
0.866727

                                      temp_min_Bilbao
```

```
0.867970
                                    temp_max_Bilbao
0.828347
                                    temp_Madrid
0.903996
                                    temp_min_Madrid
0.874548
                                    temp_max_Madrid
0.899010
                                    temp_Seville
0.841910
                                    temp_min_Seville
0.863566
                                    temp_Valencia
0.917506
                                    temp_min_Valencia
0.898616
                                    temp_max_Valencia
0.913727
temp_min_Barcelona                  temp_Barcelona
0.970264
                                    temp_max_Barcelona
0.901932
                                    temp_Bilbao
0.820751
                                    temp_min_Bilbao
0.837357
                                    temp_max_Bilbao
0.772270
                                    temp_Madrid
0.860832
                                    temp_min_Madrid
0.846014
                                    temp_max_Madrid
0.844725
                                    temp_Seville
0.777557
                                    temp_min_Seville
0.825614
                                    temp_Valencia
0.885372
                                    temp_min_Valencia
0.889764
                                    temp_max_Valencia
0.858473
temp_max_Barcelona                  temp_Barcelona
0.976904
                                    temp_min_Barcelona
0.901932
                                    temp_Bilbao
0.867023
                                    temp_min_Bilbao
0.855948
                                    temp_max_Bilbao
0.838669
                                    temp_Madrid
0.899022
                                    temp_min_Madrid
0.858746
                                    temp_max_Madrid
0.904141
                                    temp_Seville
0.855754
                                    temp_min_Seville
0.856268
                                    temp_Valencia
0.903660
                                    temp_min_Valencia
```

```
0.868367
                                    temp_max_Valencia
0.917486
temp_Bilbao                         temp_Barcelona
0.866727
                                    temp_min_Barcelona
0.820751
                                    temp_max_Barcelona
0.867023
                                    temp_min_Bilbao
0.981735
                                    temp_max_Bilbao
0.977933
                                    temp_Madrid
0.872668
                                    temp_min_Madrid
0.840910
                                    temp_max_Madrid
0.867697
                                    temp_Seville
0.821292
                                    temp_min_Seville
0.831719
                                    temp_Valencia
0.853744
                                    temp_min_Valencia
0.833132
                                    temp_max_Valencia
0.853441
temp_min_Bilbao                     temp_Barcelona
0.867970
                                    temp_min_Barcelona
0.837357
                                    temp_max_Barcelona
0.855948
                                    temp_Bilbao
0.981735
                                    temp_max_Bilbao
0.923114
                                    temp_Madrid
0.872932
                                    temp_min_Madrid
0.846695
                                    temp_max_Madrid
0.861680
                                    temp_Seville
0.807084
                                    temp_min_Seville
0.836858
                                    temp_Valencia
0.847042
                                    temp_min_Valencia
0.833425
                                    temp_max_Valencia
0.840374
temp_max_Bilbao                     temp_Barcelona
0.828347
                                    temp_min_Barcelona
0.772270
                                    temp_max_Barcelona
0.838669
                                    temp_Bilbao
0.977933
                                    temp_min_Bilbao
0.923114
                                    temp_Madrid
0.836002
                                    temp_min_Madrid
```

```
0.800589
                                    temp_max_Madrid
0.836532
                                    temp_Seville
0.798354
                                    temp_min_Seville
0.790140
                                    temp_Valencia
0.827265
                                    temp_min_Valencia
0.802560
                                    temp_max_Valencia
0.831199
temp_Madrid                         temp_Barcelona
0.903996
                                    temp_min_Barcelona
0.860832
                                    temp_max_Barcelona
0.899022
                                    temp_Bilbao
0.872668
                                    temp_min_Bilbao
0.872932
                                    temp_max_Bilbao
0.836002
                                    temp_min_Madrid
0.976940
                                    temp_max_Madrid
0.988554
                                    temp_Seville
0.909421
                                    temp_min_Seville
0.930784
                                    temp_Valencia
0.912281
                                    temp_min_Valencia
0.893334
                                    temp_max_Valencia
0.909834
temp_min_Madrid                     temp_Barcelona
0.874548
                                    temp_min_Barcelona
0.846014
                                    temp_max_Barcelona
0.858746
                                    temp_Bilbao
0.840910
                                    temp_min_Bilbao
0.846695
                                    temp_max_Bilbao
0.800589
                                    temp_Madrid
0.976940
                                    temp_max_Madrid
0.943617
                                    temp_Seville
0.869154
                                    temp_min_Seville
0.904311
                                    temp_Valencia
0.874935
                                    temp_min_Valencia
0.862004
                                    temp_max_Valencia
0.868622
temp_max_Madrid                     temp_Barcelona
0.899010
                                    temp_min_Barcelona
```

0.844725
temp_max_Barcelona
0.904141
temp_Bilbao
0.867697
temp_min_Bilbao
0.861680
temp_max_Bilbao
0.836532
temp_Madrid
0.988554
temp_min_Madrid
0.943617
temp_Seville
0.910091
temp_min_Seville
0.914793
temp_max_Seville
0.773055
temp_Valencia
0.911567
temp_min_Valencia
0.887747
temp_max_Valencia
0.913263
humidity_Madrid humidity_Seville
0.772896
temp_Seville temp_Barcelona
0.841910
temp_min_Barcelona
0.777557
temp_max_Barcelona
0.855754
temp_Bilbao
0.821292
temp_min_Bilbao
0.807084
temp_max_Bilbao
0.798354
temp_Madrid
0.909421
temp_min_Madrid
0.869154
temp_max_Madrid
0.910091
temp_min_Seville
0.957684
temp_max_Seville
0.915622
temp_Valencia
0.860792
temp_min_Valencia
0.824563
temp_max_Valencia
0.874618
temp_min_Seville temp_Barcelona
0.863566
temp_min_Barcelona
0.825614
temp_max_Barcelona
0.856268
temp_Bilbao
0.831719
temp_min_Bilbao
0.836858
temp_max_Bilbao
0.790140
temp_Madrid

```
0.930784
                                    temp_min_Madrid
0.904311
                                    temp_max_Madrid
0.914793
                                    temp_Seville
0.957684
                                    temp_max_Seville
0.763627
                                    temp_Valencia
0.874422
                                    temp_min_Valencia
0.854231
                                    temp_max_Valencia
0.873069
temp_max_Seville                    temp_max_Madrid
0.773055
                                    temp_Seville
0.915622
                                    temp_min_Seville
0.763627
                                    temp_max_Valencia
0.753980
humidity_Seville                    humidity_Madrid
0.772896
temp_Valencia                       temp_Barcelona
0.917506
                                    temp_min_Barcelona
0.885372
                                    temp_max_Barcelona
0.903660
                                    temp_Bilbao
0.853744
                                    temp_min_Bilbao
0.847042
                                    temp_max_Bilbao
0.827265
                                    temp_Madrid
0.912281
                                    temp_min_Madrid
0.874935
                                    temp_max_Madrid
0.911567
                                    temp_Seville
0.860792
                                    temp_min_Seville
0.874422
                                    temp_min_Valencia
0.987013
                                    temp_max_Valencia
0.988203
temp_min_Valencia                   temp_Barcelona
0.898616
                                    temp_min_Barcelona
0.889764
                                    temp_max_Barcelona
0.868367
                                    temp_Bilbao
0.833132
                                    temp_min_Bilbao
0.833425
                                    temp_max_Bilbao
0.802560
                                    temp_Madrid
0.893334
                                    temp_min_Madrid
0.862004
                                    temp_max_Madrid
```

```
0.887747
                                    temp_Seville
0.824563
                                    temp_min_Seville
0.854231
                                    temp_Valencia
0.987013
                                    temp_max_Valencia
0.953445
temp_max_Valencia                   temp_Barcelona
0.913727
                                    temp_min_Barcelona
0.858473
                                    temp_max_Barcelona
0.917486
                                    temp_Bilbao
0.853441
                                    temp_min_Bilbao
0.840374
                                    temp_max_Bilbao
0.831199
                                    temp_Madrid
0.909834
                                    temp_min_Madrid
0.868622
                                    temp_max_Madrid
0.913263
                                    temp_Seville
0.874618
                                    temp_min_Seville
0.873069
                                    temp_max_Seville
0.753980
                                    temp_Valencia
0.988203
                                    temp_min_Valencia
0.953445
```

What we can observe, is that apart from the amount of energy generated by fossil brown coal/lignit and hard coal, the most correlated features have to do with the temperatures among the different cities, as well the "secondary" information we have about the temperature of each individual city (i.e. the minimunm and maximum temperature).

5 3. Feature Engineering

5.1 3.1. Feature generation

The first features that we will generate for a given hour will just be that particular hour, the day and the month in which it falls into.

[63]:
```
# Generate 'hour', 'weekday' and 'month' features

for i in range(len(df_final)):
    position = df_final.index[i]
    hour = position.hour
    weekday = position.weekday()
    month = position.month
    df_final.loc[position, 'hour'] = hour
    df_final.loc[position, 'weekday'] = weekday
    df_final.loc[position, 'month'] = month
```

A very useful feature that we will generate, has to do with the business hours, i.e. whether businesses are open or not in a given hour. However, there is a certain peculiarity in Spain as the 9AM-5PM working day is not generally followed by all types of businesses, due to a lunch break (known as "siesta") in between. The most

usual business hours are from Monday-Saturday, from 9:30AM-1:30PM and then again from 4:30PM-8PM.

Therefore, the value of the 'business hour' will be equal to '2' if the given hour is within the business hours, equal to '1' if the given hour is within the "siesta" in between and equal to '0' for all other given hours.

[64]:
```
# Generate 'business hour' feature

for i in range(len(df_final)):
    position = df_final.index[i]
    hour = position.hour
    if ((hour > 8 and hour < 14) or (hour > 16 and hour < 21)):
        df_final.loc[position, 'business hour'] = 2
    elif (hour >= 14 and hour <= 16):
        df_final.loc[position, 'business hour'] = 1
    else:
        df_final.loc[position, 'business hour'] = 0
```

However, we have generated the 'business hour' feature in such a way which also includes weekends, on which fewer -or different types of- businesses are open. Thus, it is useful to also generate another feature, 'weekend', which will distuingish weekdays and weekends, as well as make a distinction between Saturdays and Sundays.

In particular, for a particular given hour, the value of 'weekend' will be equal to '0', if the hours is in a weekday, equal to '1' if the hour is in a Saturday and equal to '2' if the hour is in a Sunday.

[65]:
```
# Generate 'weekend' feature

for i in range(len(df_final)):
    position = df_final.index[i]
    weekday = position.weekday()
    if (weekday == 6):
        df_final.loc[position, 'weekday'] = 2
    elif (weekday == 5):
        df_final.loc[position, 'weekday'] = 1
    else:
        df_final.loc[position, 'weekday'] = 0
```

In order to reduce the dimensionality and potentially acquire a new kind of information, for every given hour, we will subtract the minimum temperature ('temp_min') from the maximum temperature ('temp_max') for each city and we will name that feature 'temp_range_{name of city}'.

[66]:
```
# Generate 'temp_range' for each city

cities = ['Barcelona', 'Bilbao', 'Madrid', 'Seville', 'Valencia']

for i in range(len(df_final)):
    position = df_final.index[i]
    for city in cities:
        temp_max = df_final.loc[position, 'temp_max_{}'.format(city)]
        temp_min = df_final.loc[position, 'temp_min_{}'.format(city)]
        df_final.loc[position, 'temp_range_{}'.format(city)] = abs(temp_max - temp_min)
```

After seeing that there is a high correlation among the temperatures of the different cities, we will also try creating a weighted temperature features by taking into account each city's population [7].

- Madrid: 6,155,116
- Barcelona: 5,179,243
- Valencia: 1,645,342
- Seville: 1,305,342
- Bilbao: 987,000

[67]:
```
# Calculate the weight of every city

total_pop = 6155116 + 5179243 + 1645342 + 1305342 + 987000

weight_Madrid = 6155116 / total_pop
weight_Barcelona = 5179243 / total_pop
weight_Valencia = 1645342 / total_pop
weight_Seville = 1305342 / total_pop
weight_Bilbao = 987000 / total_pop
```

[68]:
```
cities_weights = {'Madrid': weight_Madrid,
                  'Barcelona': weight_Barcelona,
                  'Valencia': weight_Valencia,
                  'Seville': weight_Seville,
                  'Bilbao': weight_Bilbao}
```

[69]:
```
# Generate 'temp_weighted' feature

for i in range(len(df_final)):
    position = df_final.index[i]
    temp_weighted = 0
    for city in cities:
        temp = df_final.loc[position, 'temp_{}'.format(city)]
        temp_weighted += temp * cities_weights.get('{}'.format(city))
    df_final.loc[position, 'temp_weighted'] = temp_weighted
```

We will also generate a new feature which aggregates both energy sources which are related to coal and are highly correlated (0.7688).

[70]:
```
df_final['generation coal all'] = df_final['generation fossil hard coal'] + df_final['generation fossil␣
↪brown coal/lignite']
```

5.2 3.2. Feature selection

[71]:
```
def multivariate_data(dataset, target, start_index, end_index, history_size,
                      target_size, step, single_step=False):
    data = []
    labels = []

    start_index = start_index + history_size
    if end_index is None:
        end_index = len(dataset) - target_size

    for i in range(start_index, end_index):
        indices = range(i-history_size, i, step)
        data.append(dataset[indices])

        if single_step:
            labels.append(target[i + target_size])
        else:
            labels.append(target[i : i + target_size])

    return np.array(data), np.array(labels)
```

[72]:
```
train_end_idx = 27048
cv_end_idx = 31056
test_end_idx = 35064
```

[73]:
```
X = df_final[df_final.columns.drop('price actual')].values
y = df_final['price actual'].values

y = y.reshape(-1, 1)
```

[74]:
```
scaler_X = MinMaxScaler(feature_range=(0, 1))
scaler_y = MinMaxScaler(feature_range=(0, 1))
```

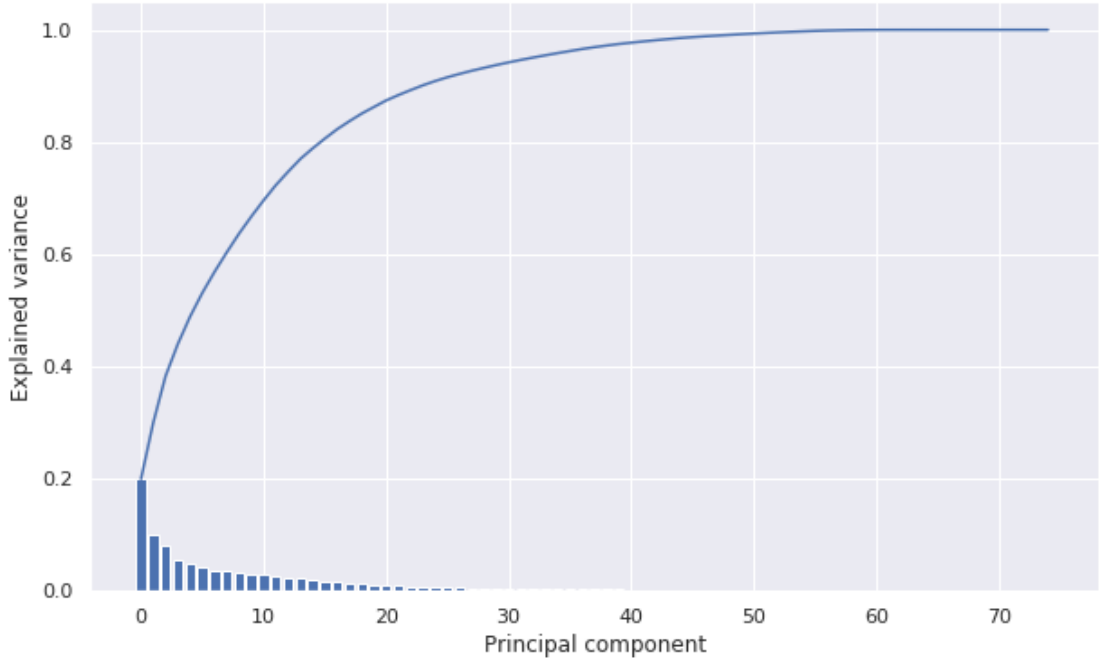
[75]:
```
scaler_X.fit(X[:train_end_idx])
scaler_y.fit(y[:train_end_idx])
```

[75]:
```
MinMaxScaler(copy=True, feature_range=(0, 1))
```

[76]:
```
X_norm = scaler_X.transform(X)
y_norm = scaler_y.transform(y)
```

[77]:
```
pca = PCA()
X_pca = pca.fit(X_norm[:train_end_idx])
```

[78]:
```
num_components = len(pca.explained_variance_ratio_)
plt.figure(figsize=(10, 6))
plt.bar(np.arange(num_components), pca.explained_variance_ratio_)
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Principal component')
plt.ylabel('Explained variance')
plt.show()
```



[79]:
```
pca = PCA(n_components=0.80)
pca.fit(X_norm[:train_end_idx])
X_pca = pca.transform(X_norm)
```

[80]:
```
X_pca.shape
```

[80]:
```
(35064, 16)
```

[81]:
```
dataset_norm = np.concatenate((X_pca, y_norm), axis=1)

past_history = 24
```

```
future_target = 0
```

[82]:
```
X_train, y_train = multivariate_data(dataset_norm, dataset_norm[:, -1],
                                     0, train_end_idx, past_history,
                                     future_target, step=1, single_step=True)
```

[83]:
```
X_val, y_val = multivariate_data(dataset_norm, dataset_norm[:, -1],
                                 train_end_idx, cv_end_idx, past_history,
                                 future_target, step=1, single_step=True)
```

[84]:
```
X_test, y_test = multivariate_data(dataset_norm, dataset_norm[:, -1],
                                   cv_end_idx, test_end_idx, past_history,
                                   future_target, step=1, single_step=True)
```

[85]:
```
batch_size = 32
buffer_size = 1000
```

[86]:
```
train = tf.data.Dataset.from_tensor_slices((X_train, y_train))
train = train.cache().shuffle(buffer_size).batch(batch_size).prefetch(1)

validation = tf.data.Dataset.from_tensor_slices((X_val, y_val))
validation = validation.batch(batch_size).prefetch(1)
```

[87]:
```
# Define some common parameters

input_shape = X_train.shape[-2:]
loss = tf.keras.losses.MeanSquaredError()
metric = [tf.keras.metrics.RootMeanSquaredError()]
lr_schedule = tf.keras.callbacks.LearningRateScheduler(
              lambda epoch: 1e-4 * 10**(epoch / 10))
early_stopping = tf.keras.callbacks.EarlyStopping(patience=10)
```

[88]:
```
y_test = y_test.reshape(-1, 1)
y_test_inv = scaler_y.inverse_transform(y_test)
```

6 4. Electricity Price Forecasting

[89]:
```
def plot_model_rmse_and_loss(history):

    # Evaluate train and validation accuracies and losses

    train_rmse = history.history['root_mean_squared_error']
    val_rmse = history.history['val_root_mean_squared_error']

    train_loss = history.history['loss']
    val_loss = history.history['val_loss']

    # Visualize epochs vs. train and validation accuracies and losses

    plt.figure(figsize=(20, 10))
    plt.subplot(1, 2, 1)
    plt.plot(train_rmse, label='Training RMSE')
    plt.plot(val_rmse, label='Validation RMSE')
    plt.legend()
    plt.title('Epochs vs. Training and Validation RMSE')

    plt.subplot(1, 2, 2)
    plt.plot(train_loss, label='Training Loss')
    plt.plot(val_loss, label='Validation Loss')
    plt.legend()
    plt.title('Epochs vs. Training and Validation Loss')
```

```
    plt.show()
```

6.1 4.1. Naive Forecast

6.2 4.2. XGBoost

[90]:
```
X_train_xgb = X_train.reshape(-1, X_train.shape[1] * X_train.shape[2])
X_val_xgb = X_val.reshape(-1, X_val.shape[1] * X_val.shape[2])
X_test_xgb = X_test.reshape(-1, X_test.shape[1] * X_test.shape[2])
```

[91]:
```
param = {'eta': 0.03, 'max_depth': 180,
         'subsample': 1.0, 'colsample_bytree': 0.95,
         'alpha': 0.1, 'lambda': 0.15, 'gamma': 0.1,
         'objective': 'reg:linear', 'eval_metric': 'rmse',
         'silent': 1, 'min_child_weight': 0.1, 'n_jobs': -1}

dtrain = xgb.DMatrix(X_train_xgb, y_train)
dval = xgb.DMatrix(X_val_xgb, y_val)
dtest = xgb.DMatrix(X_test_xgb, y_test)
eval_list = [(dtrain, 'train'), (dval, 'eval')]

xgb_model = xgb.train(param, dtrain, 180, eval_list, early_stopping_rounds=3)
```

```
[0]     train-rmse:0.144136     eval-rmse:0.117006
Multiple eval metrics have been passed: 'eval-rmse' will be used for early
stopping.

Will train until eval-rmse hasn't improved in 3 rounds.
[1]     train-rmse:0.140028     eval-rmse:0.11371
[2]     train-rmse:0.136049     eval-rmse:0.110503
[3]     train-rmse:0.132193     eval-rmse:0.107422
[4]     train-rmse:0.128457     eval-rmse:0.104417
[5]     train-rmse:0.124846     eval-rmse:0.101527
[6]     train-rmse:0.12134      eval-rmse:0.098681
[7]     train-rmse:0.117944     eval-rmse:0.095938
[8]     train-rmse:0.11466      eval-rmse:0.093297
[9]     train-rmse:0.111477     eval-rmse:0.090772
[10]    train-rmse:0.108396     eval-rmse:0.088281
[11]    train-rmse:0.105411     eval-rmse:0.085877
[12]    train-rmse:0.10252      eval-rmse:0.083553
[13]    train-rmse:0.099718     eval-rmse:0.081316
[14]    train-rmse:0.097014     eval-rmse:0.079143
[15]    train-rmse:0.094523     eval-rmse:0.077208
[16]    train-rmse:0.091977     eval-rmse:0.075181
[17]    train-rmse:0.089517     eval-rmse:0.073222
[18]    train-rmse:0.08714      eval-rmse:0.071311
[19]    train-rmse:0.084834     eval-rmse:0.069467
[20]    train-rmse:0.08261      eval-rmse:0.067703
[21]    train-rmse:0.080456     eval-rmse:0.065994
[22]    train-rmse:0.078371     eval-rmse:0.064329
[23]    train-rmse:0.076353     eval-rmse:0.06272
[24]    train-rmse:0.074412     eval-rmse:0.061201
[25]    train-rmse:0.072527     eval-rmse:0.059714
[26]    train-rmse:0.070721     eval-rmse:0.058303
[27]    train-rmse:0.068962     eval-rmse:0.056912
[28]    train-rmse:0.067353     eval-rmse:0.055754
[29]    train-rmse:0.065711     eval-rmse:0.054455
[30]    train-rmse:0.064129     eval-rmse:0.053187
[31]    train-rmse:0.062603     eval-rmse:0.051982
[32]    train-rmse:0.061123     eval-rmse:0.050814
[33]    train-rmse:0.059698     eval-rmse:0.04974
[34]    train-rmse:0.058324     eval-rmse:0.04868
[35]    train-rmse:0.057        eval-rmse:0.047662
[36]    train-rmse:0.055724     eval-rmse:0.046674
[37]    train-rmse:0.05449      eval-rmse:0.045736
[38]    train-rmse:0.053297     eval-rmse:0.04482
```

```
[39]    train-rmse:0.052149     eval-rmse:0.043885
[40]    train-rmse:0.051045     eval-rmse:0.043013
[41]    train-rmse:0.049969     eval-rmse:0.042151
[42]    train-rmse:0.048953     eval-rmse:0.04136
[43]    train-rmse:0.04797      eval-rmse:0.040612
[44]    train-rmse:0.047022     eval-rmse:0.039896
[45]    train-rmse:0.046112     eval-rmse:0.039187
[46]    train-rmse:0.045231     eval-rmse:0.03852
[47]    train-rmse:0.044392     eval-rmse:0.037906
[48]    train-rmse:0.043574     eval-rmse:0.037287
[49]    train-rmse:0.042788     eval-rmse:0.036688
[50]    train-rmse:0.042033     eval-rmse:0.036105
[51]    train-rmse:0.041305     eval-rmse:0.035563
[52]    train-rmse:0.040615     eval-rmse:0.03504
[53]    train-rmse:0.039953     eval-rmse:0.034577
[54]    train-rmse:0.03932      eval-rmse:0.034105
[55]    train-rmse:0.038707     eval-rmse:0.033673
[56]    train-rmse:0.038124     eval-rmse:0.033247
[57]    train-rmse:0.037569     eval-rmse:0.032866
[58]    train-rmse:0.037026     eval-rmse:0.032472
[59]    train-rmse:0.036501     eval-rmse:0.032098
[60]    train-rmse:0.036014     eval-rmse:0.031723
[61]    train-rmse:0.035546     eval-rmse:0.031403
[62]    train-rmse:0.035089     eval-rmse:0.031076
[63]    train-rmse:0.034664     eval-rmse:0.030807
[64]    train-rmse:0.034251     eval-rmse:0.030543
[65]    train-rmse:0.033847     eval-rmse:0.030249
[66]    train-rmse:0.033475     eval-rmse:0.029979
[67]    train-rmse:0.033111     eval-rmse:0.029714
[68]    train-rmse:0.032772     eval-rmse:0.029481
[69]    train-rmse:0.032441     eval-rmse:0.029256
[70]    train-rmse:0.032124     eval-rmse:0.029002
[71]    train-rmse:0.031821     eval-rmse:0.028811
[72]    train-rmse:0.031529     eval-rmse:0.028618
[73]    train-rmse:0.031246     eval-rmse:0.028423
[74]    train-rmse:0.03096      eval-rmse:0.028231
[75]    train-rmse:0.030707     eval-rmse:0.028047
[76]    train-rmse:0.030462     eval-rmse:0.027904
[77]    train-rmse:0.030227     eval-rmse:0.027721
[78]    train-rmse:0.029994     eval-rmse:0.027575
[79]    train-rmse:0.029764     eval-rmse:0.027443
[80]    train-rmse:0.029551     eval-rmse:0.02728
[81]    train-rmse:0.02936      eval-rmse:0.02716
[82]    train-rmse:0.029163     eval-rmse:0.027021
[83]    train-rmse:0.028962     eval-rmse:0.026865
[84]    train-rmse:0.028783     eval-rmse:0.026733
[85]    train-rmse:0.02862      eval-rmse:0.026618
[86]    train-rmse:0.028466     eval-rmse:0.026522
[87]    train-rmse:0.028321     eval-rmse:0.026424
[88]    train-rmse:0.028178     eval-rmse:0.026325
[89]    train-rmse:0.02802      eval-rmse:0.026238
[90]    train-rmse:0.027891     eval-rmse:0.026155
[91]    train-rmse:0.027751     eval-rmse:0.026059
[92]    train-rmse:0.02764      eval-rmse:0.025967
[93]    train-rmse:0.027516     eval-rmse:0.025878
[94]    train-rmse:0.027396     eval-rmse:0.025779
[95]    train-rmse:0.027262     eval-rmse:0.025691
[96]    train-rmse:0.02716      eval-rmse:0.025621
[97]    train-rmse:0.027062     eval-rmse:0.025545
[98]    train-rmse:0.026982     eval-rmse:0.025482
[99]    train-rmse:0.026873     eval-rmse:0.025427
[100]   train-rmse:0.02677      eval-rmse:0.025362
[101]   train-rmse:0.026675     eval-rmse:0.025288
[102]   train-rmse:0.026589     eval-rmse:0.025241
[103]   train-rmse:0.026517     eval-rmse:0.025194
[104]   train-rmse:0.026456     eval-rmse:0.025158
[105]   train-rmse:0.026383     eval-rmse:0.025104
[106]   train-rmse:0.026302     eval-rmse:0.025052
```

```
[107]   train-rmse:0.026231     eval-rmse:0.025
[108]   train-rmse:0.026172     eval-rmse:0.024954
[109]   train-rmse:0.02611      eval-rmse:0.024914
[110]   train-rmse:0.026041     eval-rmse:0.024873
[111]   train-rmse:0.025979     eval-rmse:0.024837
[112]   train-rmse:0.025924     eval-rmse:0.024795
[113]   train-rmse:0.025851     eval-rmse:0.024744
[114]   train-rmse:0.025793     eval-rmse:0.0247
[115]   train-rmse:0.025753     eval-rmse:0.02467
[116]   train-rmse:0.025696     eval-rmse:0.024619
[117]   train-rmse:0.02564      eval-rmse:0.024584
[118]   train-rmse:0.025598     eval-rmse:0.024546
[119]   train-rmse:0.025543     eval-rmse:0.024514
[120]   train-rmse:0.025495     eval-rmse:0.024487
[121]   train-rmse:0.025413     eval-rmse:0.024424
[122]   train-rmse:0.025372     eval-rmse:0.024398
[123]   train-rmse:0.025319     eval-rmse:0.024371
[124]   train-rmse:0.025273     eval-rmse:0.024329
[125]   train-rmse:0.025237     eval-rmse:0.024295
[126]   train-rmse:0.025204     eval-rmse:0.024273
[127]   train-rmse:0.02516      eval-rmse:0.02424
[128]   train-rmse:0.025121     eval-rmse:0.024219
[129]   train-rmse:0.025053     eval-rmse:0.024164
[130]   train-rmse:0.025013     eval-rmse:0.02414
[131]   train-rmse:0.02499      eval-rmse:0.024111
[132]   train-rmse:0.024963     eval-rmse:0.024101
[133]   train-rmse:0.024938     eval-rmse:0.024074
[134]   train-rmse:0.024901     eval-rmse:0.024046
[135]   train-rmse:0.024887     eval-rmse:0.024035
[136]   train-rmse:0.024861     eval-rmse:0.024012
[137]   train-rmse:0.02482      eval-rmse:0.02397
[138]   train-rmse:0.024793     eval-rmse:0.023938
[139]   train-rmse:0.024769     eval-rmse:0.023928
[140]   train-rmse:0.024745     eval-rmse:0.023917
[141]   train-rmse:0.024717     eval-rmse:0.023898
[142]   train-rmse:0.024693     eval-rmse:0.023887
[143]   train-rmse:0.024661     eval-rmse:0.023867
[144]   train-rmse:0.024636     eval-rmse:0.02385
[145]   train-rmse:0.024612     eval-rmse:0.023826
[146]   train-rmse:0.024591     eval-rmse:0.023815
[147]   train-rmse:0.024566     eval-rmse:0.023801
[148]   train-rmse:0.024545     eval-rmse:0.023784
[149]   train-rmse:0.024512     eval-rmse:0.023762
[150]   train-rmse:0.024501     eval-rmse:0.023751
[151]   train-rmse:0.024477     eval-rmse:0.023737
[152]   train-rmse:0.024447     eval-rmse:0.023723
[153]   train-rmse:0.024441     eval-rmse:0.023717
[154]   train-rmse:0.024411     eval-rmse:0.0237
[155]   train-rmse:0.024392     eval-rmse:0.023686
[156]   train-rmse:0.024364     eval-rmse:0.023649
[157]   train-rmse:0.024344     eval-rmse:0.023644
[158]   train-rmse:0.02432      eval-rmse:0.02363
[159]   train-rmse:0.024272     eval-rmse:0.023596
[160]   train-rmse:0.024272     eval-rmse:0.023597
[161]   train-rmse:0.024271     eval-rmse:0.023597
[162]   train-rmse:0.024271     eval-rmse:0.023598
Stopping. Best iteration:
[159]   train-rmse:0.024272     eval-rmse:0.023596
```

[92]:
```
forecast = xgb_model.predict(dtest)
xgb_forecast = forecast.reshape(-1, 1)

xgb_forecast_inv = scaler_y.inverse_transform(xgb_forecast)

rmse_xgb = sqrt(mean_squared_error(y_test_inv, xgb_forecast_inv))
print('RMSE of hour-ahead electricity price XGBoost forecast: {}'
```

```
      .format(round(rmse_xgb, 3)))
```

```
RMSE of hour-ahead electricity price XGBoost forecast: 2.214
```

6.3 4.3. LSTM

[93]:
```
tf.keras.backend.clear_session()

multivariate_lstm = tf.keras.models.Sequential([
    LSTM(100, input_shape=input_shape,
         return_sequences=True),
    Flatten(),
    Dense(200, activation='relu'),
    Dropout(0.1),
    Dense(1)
])

model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
                   'multivariate_lstm.h5', monitor=('val_loss'), save_best_only=True)
optimizer = tf.keras.optimizers.Adam(lr=6e-3, amsgrad=True)

multivariate_lstm.compile(loss=loss,
                          optimizer=optimizer,
                          metrics=metric)
```

[94]:
```
history = multivariate_lstm.fit(train, epochs=120,
                                validation_data=validation,
                                callbacks=[early_stopping,
                                           model_checkpoint])
```

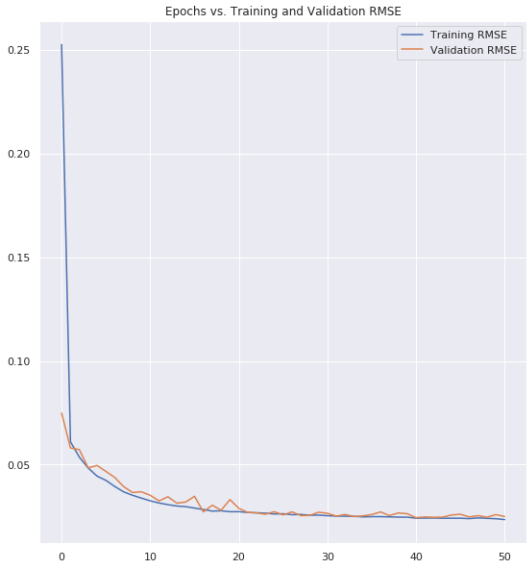
```
Epoch 1/120
845/845 [==============================] - 10s 12ms/step - loss: 0.0638 -
root_mean_squared_error: 0.2526 - val_loss: 0.0000e+00 -
val_root_mean_squared_error: 0.0000e+00
Epoch 2/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0037 -
root_mean_squared_error: 0.0610 - val_loss: 0.0034 -
val_root_mean_squared_error: 0.0580
Epoch 3/120
845/845 [==============================] - 4s 5ms/step - loss: 0.0029 -
root_mean_squared_error: 0.0536 - val_loss: 0.0033 -
val_root_mean_squared_error: 0.0572
Epoch 4/120
845/845 [==============================] - 5s 5ms/step - loss: 0.0023 -
root_mean_squared_error: 0.0484 - val_loss: 0.0023 -
val_root_mean_squared_error: 0.0484
Epoch 5/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0020 -
root_mean_squared_error: 0.0445 - val_loss: 0.0025 -
val_root_mean_squared_error: 0.0496
Epoch 6/120
845/845 [==============================] - 5s 5ms/step - loss: 0.0018 -
root_mean_squared_error: 0.0424 - val_loss: 0.0022 -
val_root_mean_squared_error: 0.0467
Epoch 7/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0016 -
root_mean_squared_error: 0.0394 - val_loss: 0.0019 -
val_root_mean_squared_error: 0.0438
Epoch 8/120
845/845 [==============================] - 5s 5ms/step - loss: 0.0014 -
root_mean_squared_error: 0.0369 - val_loss: 0.0015 -
val_root_mean_squared_error: 0.0394
Epoch 9/120
845/845 [==============================] - 4s 5ms/step - loss: 0.0012 -
root_mean_squared_error: 0.0352 - val_loss: 0.0013 -
val_root_mean_squared_error: 0.0366
```

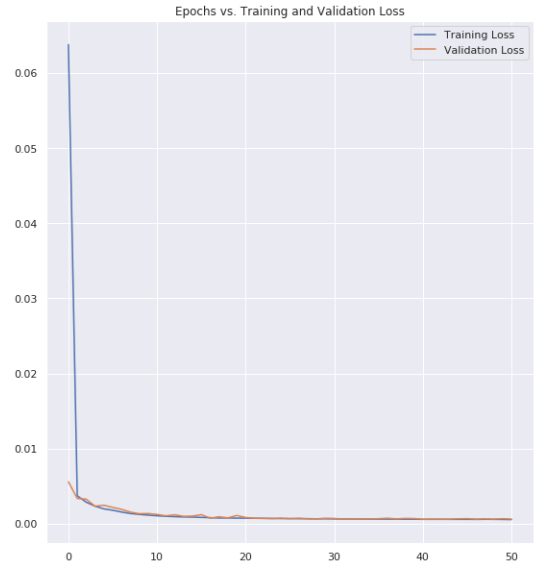
```
Epoch 10/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0011 -
root_mean_squared_error: 0.0338 - val_loss: 0.0014 -
val_root_mean_squared_error: 0.0369
Epoch 11/120
845/845 [==============================] - 4s 5ms/step - loss: 0.0011 -
root_mean_squared_error: 0.0325 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0352
Epoch 12/120
845/845 [==============================] - 5s 6ms/step - loss: 9.9125e-04 -
root_mean_squared_error: 0.0315 - val_loss: 0.0011 -
val_root_mean_squared_error: 0.0325
Epoch 13/120
845/845 [==============================] - 4s 5ms/step - loss: 9.4183e-04 -
root_mean_squared_error: 0.0307 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0345
Epoch 14/120
845/845 [==============================] - 5s 6ms/step - loss: 9.0184e-04 -
root_mean_squared_error: 0.0300 - val_loss: 9.8679e-04 -
val_root_mean_squared_error: 0.0314
Epoch 15/120
845/845 [==============================] - 4s 5ms/step - loss: 8.8388e-04 -
root_mean_squared_error: 0.0297 - val_loss: 0.0010 -
val_root_mean_squared_error: 0.0319
Epoch 16/120
845/845 [==============================] - 4s 5ms/step - loss: 8.4302e-04 -
root_mean_squared_error: 0.0290 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0347
Epoch 17/120
845/845 [==============================] - 5s 6ms/step - loss: 8.0231e-04 -
root_mean_squared_error: 0.0283 - val_loss: 7.3827e-04 -
val_root_mean_squared_error: 0.0272
Epoch 18/120
845/845 [==============================] - 4s 5ms/step - loss: 7.6129e-04 -
root_mean_squared_error: 0.0276 - val_loss: 9.2292e-04 -
val_root_mean_squared_error: 0.0304
Epoch 19/120
845/845 [==============================] - 5s 6ms/step - loss: 7.6804e-04 -
root_mean_squared_error: 0.0277 - val_loss: 7.8086e-04 -
val_root_mean_squared_error: 0.0280
Epoch 20/120
845/845 [==============================] - 4s 5ms/step - loss: 7.4438e-04 -
root_mean_squared_error: 0.0273 - val_loss: 0.0011 -
val_root_mean_squared_error: 0.0331
Epoch 21/120
845/845 [==============================] - 5s 6ms/step - loss: 7.4567e-04 -
root_mean_squared_error: 0.0273 - val_loss: 8.3363e-04 -
val_root_mean_squared_error: 0.0289
Epoch 22/120
845/845 [==============================] - 4s 5ms/step - loss: 7.2496e-04 -
root_mean_squared_error: 0.0269 - val_loss: 7.2440e-04 -
val_root_mean_squared_error: 0.0270
Epoch 23/120
845/845 [==============================] - 5s 6ms/step - loss: 7.1686e-04 -
root_mean_squared_error: 0.0268 - val_loss: 7.1168e-04 -
val_root_mean_squared_error: 0.0267
Epoch 24/120
845/845 [==============================] - 5s 6ms/step - loss: 7.0561e-04 -
root_mean_squared_error: 0.0266 - val_loss: 6.7853e-04 -
val_root_mean_squared_error: 0.0261
Epoch 25/120
845/845 [==============================] - 4s 5ms/step - loss: 6.8698e-04 -
root_mean_squared_error: 0.0262 - val_loss: 7.4224e-04 -
val_root_mean_squared_error: 0.0273
Epoch 26/120
845/845 [==============================] - 5s 6ms/step - loss: 6.9129e-04 -
root_mean_squared_error: 0.0263 - val_loss: 6.6180e-04 -
val_root_mean_squared_error: 0.0258
```

```
Epoch 27/120
845/845 [==============================] - 4s 5ms/step - loss: 6.6771e-04 -
root_mean_squared_error: 0.0258 - val_loss: 7.3824e-04 -
val_root_mean_squared_error: 0.0272
Epoch 28/120
845/845 [==============================] - 5s 6ms/step - loss: 6.7459e-04 -
root_mean_squared_error: 0.0260 - val_loss: 6.4324e-04 -
val_root_mean_squared_error: 0.0254
Epoch 29/120
845/845 [==============================] - 5s 5ms/step - loss: 6.5291e-04 -
root_mean_squared_error: 0.0256 - val_loss: 6.4672e-04 -
val_root_mean_squared_error: 0.0255
Epoch 30/120
845/845 [==============================] - 5s 5ms/step - loss: 6.5709e-04 -
root_mean_squared_error: 0.0257 - val_loss: 7.2976e-04 -
val_root_mean_squared_error: 0.0270
Epoch 31/120
845/845 [==============================] - 5s 6ms/step - loss: 6.4289e-04 -
root_mean_squared_error: 0.0254 - val_loss: 7.0110e-04 -
val_root_mean_squared_error: 0.0265
Epoch 32/120
845/845 [==============================] - 5s 5ms/step - loss: 6.3336e-04 -
root_mean_squared_error: 0.0252 - val_loss: 6.3071e-04 -
val_root_mean_squared_error: 0.0252
Epoch 33/120
845/845 [==============================] - 5s 6ms/step - loss: 6.3111e-04 -
root_mean_squared_error: 0.0251 - val_loss: 6.7171e-04 -
val_root_mean_squared_error: 0.0260
Epoch 34/120
845/845 [==============================] - 5s 5ms/step - loss: 6.2780e-04 -
root_mean_squared_error: 0.0251 - val_loss: 6.1929e-04 -
val_root_mean_squared_error: 0.0249
Epoch 35/120
845/845 [==============================] - 5s 6ms/step - loss: 6.1499e-04 -
root_mean_squared_error: 0.0248 - val_loss: 6.3644e-04 -
val_root_mean_squared_error: 0.0253
Epoch 36/120
845/845 [==============================] - 5s 5ms/step - loss: 6.2039e-04 -
root_mean_squared_error: 0.0249 - val_loss: 6.6956e-04 -
val_root_mean_squared_error: 0.0259
Epoch 37/120
845/845 [==============================] - 5s 5ms/step - loss: 6.2012e-04 -
root_mean_squared_error: 0.0249 - val_loss: 7.3612e-04 -
val_root_mean_squared_error: 0.0272
Epoch 38/120
845/845 [==============================] - 5s 6ms/step - loss: 6.1510e-04 -
root_mean_squared_error: 0.0248 - val_loss: 6.4498e-04 -
val_root_mean_squared_error: 0.0254
Epoch 39/120
845/845 [==============================] - 5s 5ms/step - loss: 6.0882e-04 -
root_mean_squared_error: 0.0247 - val_loss: 7.0982e-04 -
val_root_mean_squared_error: 0.0267
Epoch 40/120
845/845 [==============================] - 5s 6ms/step - loss: 6.0812e-04 -
root_mean_squared_error: 0.0247 - val_loss: 6.9143e-04 -
val_root_mean_squared_error: 0.0263
Epoch 41/120
845/845 [==============================] - 5s 5ms/step - loss: 5.8467e-04 -
root_mean_squared_error: 0.0242 - val_loss: 5.9466e-04 -
val_root_mean_squared_error: 0.0244
Epoch 42/120
845/845 [==============================] - 5s 6ms/step - loss: 5.8574e-04 -
root_mean_squared_error: 0.0242 - val_loss: 6.1145e-04 -
val_root_mean_squared_error: 0.0248
Epoch 43/120
845/845 [==============================] - 5s 5ms/step - loss: 5.8894e-04 -
root_mean_squared_error: 0.0243 - val_loss: 6.0271e-04 -
val_root_mean_squared_error: 0.0246
```

```
Epoch 44/120
845/845 [==============================] - 4s 5ms/step - loss: 5.8250e-04 -
root_mean_squared_error: 0.0241 - val_loss: 6.0717e-04 -
val_root_mean_squared_error: 0.0247
Epoch 45/120
845/845 [==============================] - 5s 6ms/step - loss: 5.8294e-04 -
root_mean_squared_error: 0.0241 - val_loss: 6.5797e-04 -
val_root_mean_squared_error: 0.0257
Epoch 46/120
845/845 [==============================] - 4s 5ms/step - loss: 5.8310e-04 -
root_mean_squared_error: 0.0241 - val_loss: 6.7807e-04 -
val_root_mean_squared_error: 0.0261
Epoch 47/120
845/845 [==============================] - 5s 6ms/step - loss: 5.7392e-04 -
root_mean_squared_error: 0.0240 - val_loss: 6.1184e-04 -
val_root_mean_squared_error: 0.0248
Epoch 48/120
845/845 [==============================] - 4s 5ms/step - loss: 5.9138e-04 -
root_mean_squared_error: 0.0243 - val_loss: 6.4292e-04 -
val_root_mean_squared_error: 0.0254
Epoch 49/120
845/845 [==============================] - 5s 6ms/step - loss: 5.8013e-04 -
root_mean_squared_error: 0.0241 - val_loss: 6.0485e-04 -
val_root_mean_squared_error: 0.0246
Epoch 50/120
845/845 [==============================] - 5s 6ms/step - loss: 5.7181e-04 -
root_mean_squared_error: 0.0239 - val_loss: 6.6748e-04 -
val_root_mean_squared_error: 0.0259
Epoch 51/120
845/845 [==============================] - 4s 5ms/step - loss: 5.5187e-04 -
root_mean_squared_error: 0.0235 - val_loss: 6.2328e-04 -
val_root_mean_squared_error: 0.0250
```

[95]:
```
plot_model_rmse_and_loss(history)
```





[96]:
```
multivariate_lstm = tf.keras.models.load_model('multivariate_lstm.h5')

forecast = multivariate_lstm.predict(X_test)
lstm_forecast = scaler_y.inverse_transform(forecast)

rmse_lstm = sqrt(mean_squared_error(y_test_inv,
```

```
                                    lstm_forecast))
print('RMSE of hour-ahead electricity price LSTM forecast: {}'
      .format(round(rmse_lstm, 3)))
```

```
RMSE of hour-ahead electricity price LSTM forecast: 2.265
```

6.4 4.4. Stacked LSTM

[97]:
```
tf.keras.backend.clear_session()

multivariate_stacked_lstm = tf.keras.models.Sequential([
    LSTM(250, input_shape=input_shape,
         return_sequences=True),
    LSTM(150, return_sequences=True),
    Flatten(),
    Dense(150, activation='relu'),
    Dropout(0.1),
    Dense(1)
])

model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
                   'multivariate_stacked_lstm.h5', save_best_only=True)
optimizer = tf.keras.optimizers.Adam(lr=3e-3, amsgrad=True)

multivariate_stacked_lstm.compile(loss=loss,
                                  optimizer=optimizer,
                                  metrics=metric)
```

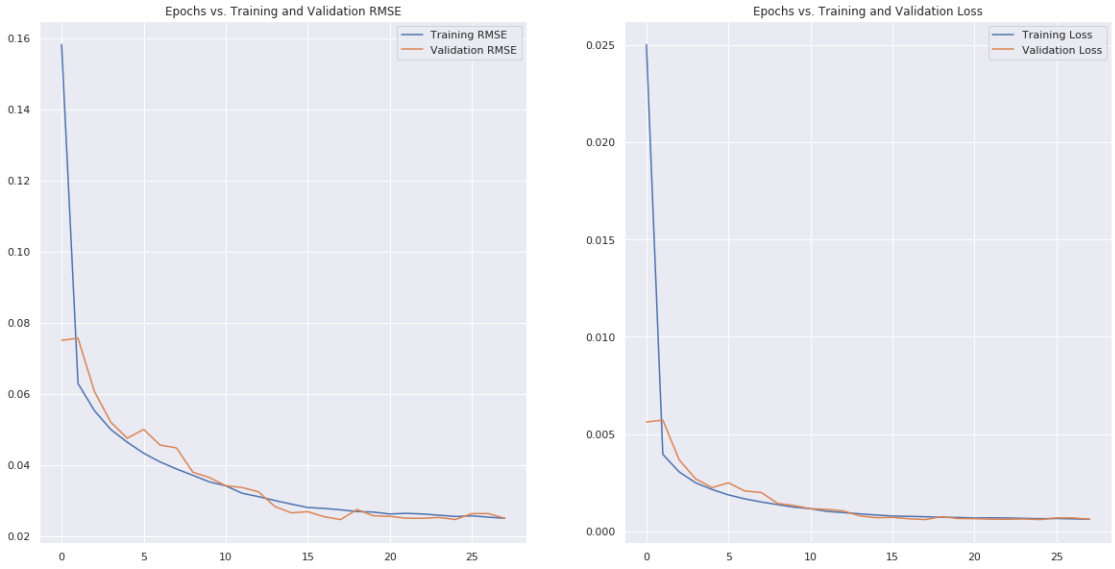
[98]:
```
history = multivariate_stacked_lstm.fit(train, epochs=120,
                                validation_data=validation,
                                callbacks=[early_stopping,
                                           model_checkpoint])
```

```
Epoch 1/120
845/845 [==============================] - 11s 13ms/step - loss: 0.0250 -
root_mean_squared_error: 0.1581 - val_loss: 0.0000e+00 -
val_root_mean_squared_error: 0.0000e+00
Epoch 2/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0040 -
root_mean_squared_error: 0.0629 - val_loss: 0.0057 -
val_root_mean_squared_error: 0.0756
Epoch 3/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0031 -
root_mean_squared_error: 0.0552 - val_loss: 0.0037 -
val_root_mean_squared_error: 0.0606
Epoch 4/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0025 -
root_mean_squared_error: 0.0500 - val_loss: 0.0027 -
val_root_mean_squared_error: 0.0520
Epoch 5/120
845/845 [==============================] - 6s 8ms/step - loss: 0.0022 -
root_mean_squared_error: 0.0464 - val_loss: 0.0022 -
val_root_mean_squared_error: 0.0475
Epoch 6/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0019 -
root_mean_squared_error: 0.0433 - val_loss: 0.0025 -
val_root_mean_squared_error: 0.0500
Epoch 7/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0017 -
root_mean_squared_error: 0.0409 - val_loss: 0.0021 -
val_root_mean_squared_error: 0.0456
Epoch 8/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0015 -
root_mean_squared_error: 0.0389 - val_loss: 0.0020 -
val_root_mean_squared_error: 0.0448
Epoch 9/120
```

```
845/845 [==============================] - 7s 8ms/step - loss: 0.0014 -
root_mean_squared_error: 0.0371 - val_loss: 0.0014 -
val_root_mean_squared_error: 0.0380
Epoch 10/120
845/845 [==============================] - 6s 8ms/step - loss: 0.0012 -
root_mean_squared_error: 0.0353 - val_loss: 0.0013 -
val_root_mean_squared_error: 0.0365
Epoch 11/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0012 -
root_mean_squared_error: 0.0342 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0341
Epoch 12/120
845/845 [==============================] - 7s 8ms/step - loss: 0.0010 -
root_mean_squared_error: 0.0320 - val_loss: 0.0011 -
val_root_mean_squared_error: 0.0337
Epoch 13/120
845/845 [==============================] - 7s 8ms/step - loss: 9.6706e-04 -
root_mean_squared_error: 0.0311 - val_loss: 0.0011 -
val_root_mean_squared_error: 0.0325
Epoch 14/120
845/845 [==============================] - 7s 8ms/step - loss: 9.0005e-04 -
root_mean_squared_error: 0.0300 - val_loss: 7.9840e-04 -
val_root_mean_squared_error: 0.0283
Epoch 15/120
845/845 [==============================] - 6s 8ms/step - loss: 8.4043e-04 -
root_mean_squared_error: 0.0290 - val_loss: 7.0175e-04 -
val_root_mean_squared_error: 0.0265
Epoch 16/120
845/845 [==============================] - 7s 8ms/step - loss: 7.8642e-04 -
root_mean_squared_error: 0.0280 - val_loss: 7.1915e-04 -
val_root_mean_squared_error: 0.0269
Epoch 17/120
845/845 [==============================] - 7s 8ms/step - loss: 7.7193e-04 -
root_mean_squared_error: 0.0278 - val_loss: 6.4572e-04 -
val_root_mean_squared_error: 0.0255
Epoch 18/120
845/845 [==============================] - 7s 8ms/step - loss: 7.5000e-04 -
root_mean_squared_error: 0.0274 - val_loss: 6.0547e-04 -
val_root_mean_squared_error: 0.0246
Epoch 19/120
845/845 [==============================] - 7s 8ms/step - loss: 7.2431e-04 -
root_mean_squared_error: 0.0269 - val_loss: 7.5272e-04 -
val_root_mean_squared_error: 0.0275
Epoch 20/120
845/845 [==============================] - 6s 8ms/step - loss: 7.1571e-04 -
root_mean_squared_error: 0.0268 - val_loss: 6.5952e-04 -
val_root_mean_squared_error: 0.0257
Epoch 21/120
845/845 [==============================] - 7s 8ms/step - loss: 6.8698e-04 -
root_mean_squared_error: 0.0262 - val_loss: 6.5110e-04 -
val_root_mean_squared_error: 0.0256
Epoch 22/120
845/845 [==============================] - 7s 8ms/step - loss: 6.9635e-04 -
root_mean_squared_error: 0.0264 - val_loss: 6.2491e-04 -
val_root_mean_squared_error: 0.0250
Epoch 23/120
845/845 [==============================] - 7s 8ms/step - loss: 6.8794e-04 -
root_mean_squared_error: 0.0262 - val_loss: 6.2290e-04 -
val_root_mean_squared_error: 0.0250
Epoch 24/120
845/845 [==============================] - 7s 8ms/step - loss: 6.6925e-04 -
root_mean_squared_error: 0.0259 - val_loss: 6.3555e-04 -
val_root_mean_squared_error: 0.0253
Epoch 25/120
845/845 [==============================] - 6s 7ms/step - loss: 6.5141e-04 -
root_mean_squared_error: 0.0255 - val_loss: 6.0574e-04 -
val_root_mean_squared_error: 0.0247
Epoch 26/120
```

```
845/845 [==============================] - 7s 8ms/step - loss: 6.6008e-04 -
root_mean_squared_error: 0.0257 - val_loss: 6.9070e-04 -
val_root_mean_squared_error: 0.0263
Epoch 27/120
845/845 [==============================] - 6s 8ms/step - loss: 6.4147e-04 -
root_mean_squared_error: 0.0253 - val_loss: 6.9111e-04 -
val_root_mean_squared_error: 0.0263
Epoch 28/120
845/845 [==============================] - 7s 8ms/step - loss: 6.2534e-04 -
root_mean_squared_error: 0.0250 - val_loss: 6.2449e-04 -
val_root_mean_squared_error: 0.0250
```

[99]:
```
plot_model_rmse_and_loss(history)
```



[100]:
```
multivariate_stacked_lstm = tf.keras.models.load_model('multivariate_stacked_lstm.h5')

forecast = multivariate_stacked_lstm.predict(X_test)
multivariate_stacked_lstm_forecast = scaler_y.inverse_transform(forecast)

rmse_mult_stacked_lstm = sqrt(mean_squared_error(y_test_inv,
                                                 multivariate_stacked_lstm_forecast))
print('RMSE of hour-ahead electricity price multivariate Stacked LSTM forecast: {}'
      .format(round(rmse_mult_stacked_lstm, 3)))
```

```
RMSE of hour-ahead electricity price multivariate Stacked LSTM forecast: 2.348
```

6.5 4.5. CNN

[101]:
```
tf.keras.backend.clear_session()

multivariate_cnn = tf.keras.models.Sequential([
    Conv1D(filters=48, kernel_size=2,
           strides=1, padding='causal',
           activation='relu',
           input_shape=input_shape),
    Flatten(),
    Dense(48, activation='relu'),
    Dense(1)
])
```

```
model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
                   'multivariate_cnn.h5', save_best_only=True)
optimizer = tf.keras.optimizers.Adam(lr=6e-3, amsgrad=True)

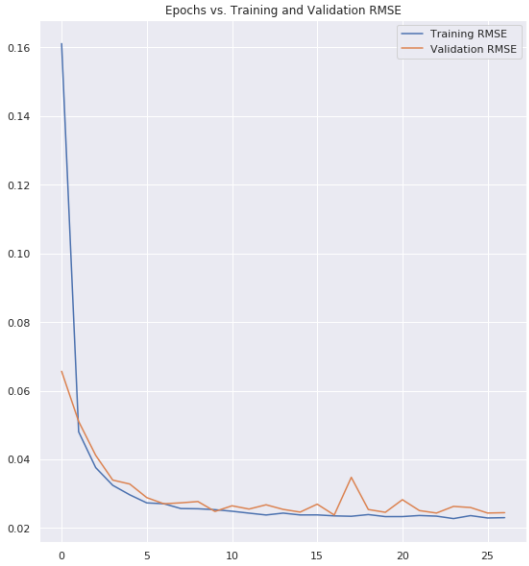
multivariate_cnn.compile(loss=loss,
                         optimizer=optimizer,
                         metrics=metric)
```

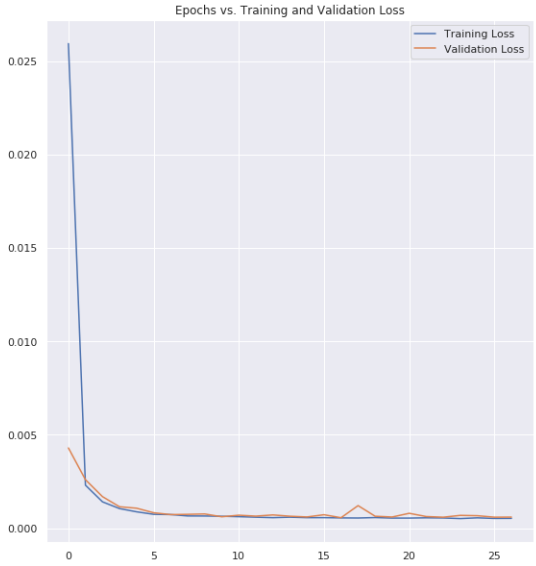
[102]:
```
history = multivariate_cnn.fit(train, epochs=120,
                               validation_data=validation,
                               callbacks=[early_stopping,
                                          model_checkpoint])
```

```
Epoch 1/120
845/845 [==============================] - 5s 5ms/step - loss: 0.0259 -
root_mean_squared_error: 0.1610 - val_loss: 0.0000e+00 -
val_root_mean_squared_error: 0.0000e+00
Epoch 2/120
845/845 [==============================] - 3s 3ms/step - loss: 0.0023 -
root_mean_squared_error: 0.0480 - val_loss: 0.0026 -
val_root_mean_squared_error: 0.0510
Epoch 3/120
845/845 [==============================] - 3s 4ms/step - loss: 0.0014 -
root_mean_squared_error: 0.0376 - val_loss: 0.0017 -
val_root_mean_squared_error: 0.0412
Epoch 4/120
845/845 [==============================] - 3s 4ms/step - loss: 0.0011 -
root_mean_squared_error: 0.0325 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0340
Epoch 5/120
845/845 [==============================] - 3s 4ms/step - loss: 8.8127e-04 -
root_mean_squared_error: 0.0297 - val_loss: 0.0011 -
val_root_mean_squared_error: 0.0328
Epoch 6/120
845/845 [==============================] - 3s 4ms/step - loss: 7.4647e-04 -
root_mean_squared_error: 0.0273 - val_loss: 8.3136e-04 -
val_root_mean_squared_error: 0.0289
Epoch 7/120
845/845 [==============================] - 3s 4ms/step - loss: 7.3408e-04 -
root_mean_squared_error: 0.0271 - val_loss: 7.3198e-04 -
val_root_mean_squared_error: 0.0271
Epoch 8/120
845/845 [==============================] - 3s 3ms/step - loss: 6.5996e-04 -
root_mean_squared_error: 0.0257 - val_loss: 7.4612e-04 -
val_root_mean_squared_error: 0.0274
Epoch 9/120
845/845 [==============================] - 3s 3ms/step - loss: 6.5657e-04 -
root_mean_squared_error: 0.0256 - val_loss: 7.6647e-04 -
val_root_mean_squared_error: 0.0277
Epoch 10/120
845/845 [==============================] - 3s 3ms/step - loss: 6.4303e-04 -
root_mean_squared_error: 0.0254 - val_loss: 6.1548e-04 -
val_root_mean_squared_error: 0.0248
Epoch 11/120
845/845 [==============================] - 3s 4ms/step - loss: 6.2050e-04 -
root_mean_squared_error: 0.0249 - val_loss: 6.9994e-04 -
val_root_mean_squared_error: 0.0265
Epoch 12/120
845/845 [==============================] - 3s 3ms/step - loss: 5.9252e-04 -
root_mean_squared_error: 0.0244 - val_loss: 6.5010e-04 -
val_root_mean_squared_error: 0.0255
Epoch 13/120
845/845 [==============================] - 3s 3ms/step - loss: 5.6691e-04 -
root_mean_squared_error: 0.0238 - val_loss: 7.1474e-04 -
val_root_mean_squared_error: 0.0268
Epoch 14/120
845/845 [==============================] - 3s 4ms/step - loss: 5.9321e-04 -
```

```
root_mean_squared_error: 0.0244 - val_loss: 6.4531e-04 -
val_root_mean_squared_error: 0.0254
Epoch 15/120
845/845 [==============================] - 3s 4ms/step - loss: 5.6769e-04 -
root_mean_squared_error: 0.0238 - val_loss: 6.0602e-04 -
val_root_mean_squared_error: 0.0247
Epoch 16/120
845/845 [==============================] - 3s 3ms/step - loss: 5.6842e-04 -
root_mean_squared_error: 0.0238 - val_loss: 7.2486e-04 -
val_root_mean_squared_error: 0.0270
Epoch 17/120
845/845 [==============================] - 3s 3ms/step - loss: 5.5477e-04 -
root_mean_squared_error: 0.0236 - val_loss: 5.6516e-04 -
val_root_mean_squared_error: 0.0238
Epoch 18/120
845/845 [==============================] - 3s 4ms/step - loss: 5.4937e-04 -
root_mean_squared_error: 0.0234 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0348
Epoch 19/120
845/845 [==============================] - 3s 3ms/step - loss: 5.7297e-04 -
root_mean_squared_error: 0.0239 - val_loss: 6.4459e-04 -
val_root_mean_squared_error: 0.0254
Epoch 20/120
845/845 [==============================] - 3s 3ms/step - loss: 5.4483e-04 -
root_mean_squared_error: 0.0233 - val_loss: 6.0315e-04 -
val_root_mean_squared_error: 0.0246
Epoch 21/120
845/845 [==============================] - 3s 3ms/step - loss: 5.4523e-04 -
root_mean_squared_error: 0.0233 - val_loss: 8.0104e-04 -
val_root_mean_squared_error: 0.0283
Epoch 22/120
845/845 [==============================] - 3s 4ms/step - loss: 5.5992e-04 -
root_mean_squared_error: 0.0237 - val_loss: 6.2811e-04 -
val_root_mean_squared_error: 0.0251
Epoch 23/120
845/845 [==============================] - 3s 3ms/step - loss: 5.5113e-04 -
root_mean_squared_error: 0.0235 - val_loss: 5.9244e-04 -
val_root_mean_squared_error: 0.0244
Epoch 24/120
845/845 [==============================] - 3s 3ms/step - loss: 5.1827e-04 -
root_mean_squared_error: 0.0228 - val_loss: 6.9012e-04 -
val_root_mean_squared_error: 0.0263
Epoch 25/120
845/845 [==============================] - 3s 4ms/step - loss: 5.5822e-04 -
root_mean_squared_error: 0.0236 - val_loss: 6.7299e-04 -
val_root_mean_squared_error: 0.0260
Epoch 26/120
845/845 [==============================] - 3s 4ms/step - loss: 5.2615e-04 -
root_mean_squared_error: 0.0229 - val_loss: 5.9357e-04 -
val_root_mean_squared_error: 0.0244
Epoch 27/120
845/845 [==============================] - 3s 3ms/step - loss: 5.3070e-04 -
root_mean_squared_error: 0.0230 - val_loss: 5.9797e-04 -
val_root_mean_squared_error: 0.0245
```

[103]:
```
plot_model_rmse_and_loss(history)
```





[104]:
```
multivariate_cnn = tf.keras.models.load_model('multivariate_cnn.h5')

forecast = multivariate_cnn.predict(X_test)
multivariate_cnn_forecast = scaler_y.inverse_transform(forecast)

rmse_mult_cnn = sqrt(mean_squared_error(y_test_inv,
                                        multivariate_cnn_forecast))
print('RMSE of hour-ahead electricity price multivariate CNN forecast: {}'
      .format(round(rmse_mult_cnn, 3)))
```

```
RMSE of hour-ahead electricity price multivariate CNN forecast: 2.324
```

6.6 4.6. CNN-LSTM

[105]:
```
tf.keras.backend.clear_session()

multivariate_cnn_lstm = tf.keras.models.Sequential([
    Conv1D(filters=100, kernel_size=2,
           strides=1, padding='causal',
           activation='relu',
           input_shape=input_shape),
    LSTM(100, return_sequences=True),
    Flatten(),
    Dense(50, activation='relu'),
    Dense(1)
])

model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
                   'multivariate_cnn_lstm.h5', save_best_only=True)
optimizer = tf.keras.optimizers.Adam(lr=4e-3, amsgrad=True)

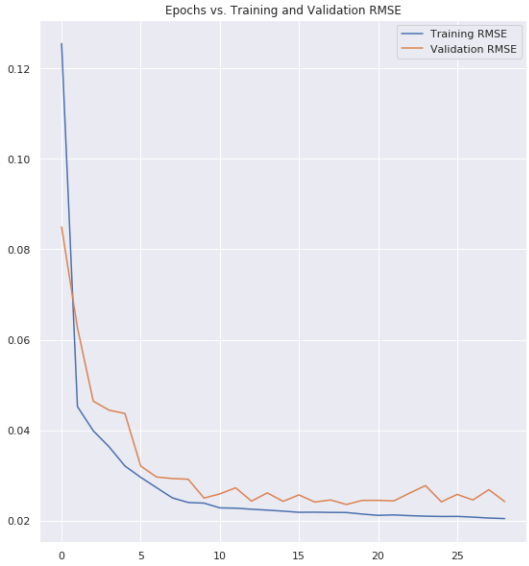
multivariate_cnn_lstm.compile(loss=loss,
                          optimizer=optimizer,
                          metrics=metric)
```

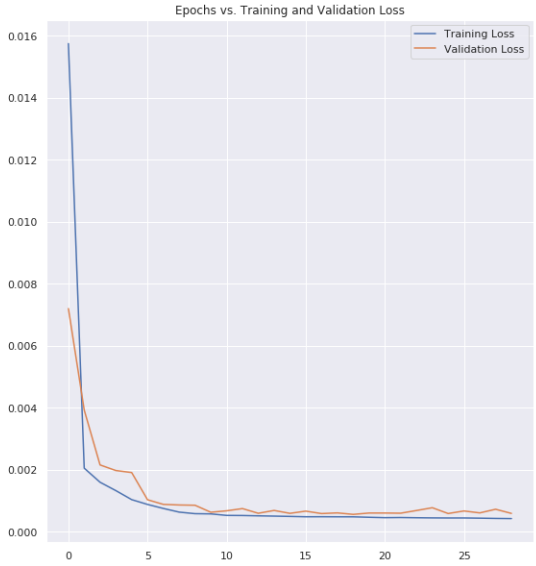
[106]:
```
history = multivariate_cnn_lstm.fit(train, epochs=120,
                                    validation_data=validation,
                                    callbacks=[early_stopping,
                                               model_checkpoint])
```

```
Epoch 1/120
845/845 [==============================] - 8s 10ms/step - loss: 0.0157 -
root_mean_squared_error: 0.1255 - val_loss: 0.0000e+00 -
val_root_mean_squared_error: 0.0000e+00
Epoch 2/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0020 -
root_mean_squared_error: 0.0452 - val_loss: 0.0039 -
val_root_mean_squared_error: 0.0627
Epoch 3/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0016 -
root_mean_squared_error: 0.0399 - val_loss: 0.0022 -
val_root_mean_squared_error: 0.0464
Epoch 4/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0013 -
root_mean_squared_error: 0.0364 - val_loss: 0.0020 -
val_root_mean_squared_error: 0.0444
Epoch 5/120
845/845 [==============================] - 6s 7ms/step - loss: 0.0010 -
root_mean_squared_error: 0.0321 - val_loss: 0.0019 -
val_root_mean_squared_error: 0.0437
Epoch 6/120
845/845 [==============================] - 5s 6ms/step - loss: 8.7611e-04 -
root_mean_squared_error: 0.0296 - val_loss: 0.0010 -
val_root_mean_squared_error: 0.0321
Epoch 7/120
845/845 [==============================] - 5s 6ms/step - loss: 7.4629e-04 -
root_mean_squared_error: 0.0273 - val_loss: 8.7819e-04 -
val_root_mean_squared_error: 0.0297
Epoch 8/120
845/845 [==============================] - 5s 6ms/step - loss: 6.2798e-04 -
root_mean_squared_error: 0.0251 - val_loss: 8.5845e-04 -
val_root_mean_squared_error: 0.0293
Epoch 9/120
845/845 [==============================] - 5s 6ms/step - loss: 5.7808e-04 -
root_mean_squared_error: 0.0240 - val_loss: 8.5006e-04 -
val_root_mean_squared_error: 0.0292
Epoch 10/120
845/845 [==============================] - 5s 6ms/step - loss: 5.7134e-04 -
root_mean_squared_error: 0.0239 - val_loss: 6.2409e-04 -
val_root_mean_squared_error: 0.0250
Epoch 11/120
845/845 [==============================] - 5s 6ms/step - loss: 5.2220e-04 -
root_mean_squared_error: 0.0229 - val_loss: 6.7068e-04 -
val_root_mean_squared_error: 0.0259
Epoch 12/120
845/845 [==============================] - 5s 6ms/step - loss: 5.1906e-04 -
root_mean_squared_error: 0.0228 - val_loss: 7.4205e-04 -
val_root_mean_squared_error: 0.0273
Epoch 13/120
845/845 [==============================] - 5s 6ms/step - loss: 5.0827e-04 -
root_mean_squared_error: 0.0226 - val_loss: 5.9088e-04 -
val_root_mean_squared_error: 0.0243
Epoch 14/120
845/845 [==============================] - 5s 6ms/step - loss: 4.9910e-04 -
root_mean_squared_error: 0.0224 - val_loss: 6.8214e-04 -
val_root_mean_squared_error: 0.0262
Epoch 15/120
845/845 [==============================] - 5s 6ms/step - loss: 4.8971e-04 -
root_mean_squared_error: 0.0221 - val_loss: 5.8872e-04 -
val_root_mean_squared_error: 0.0243
Epoch 16/120
845/845 [==============================] - 5s 6ms/step - loss: 4.7825e-04 -
root_mean_squared_error: 0.0219 - val_loss: 6.5966e-04 -
val_root_mean_squared_error: 0.0257
Epoch 17/120
845/845 [==============================] - 5s 6ms/step - loss: 4.7957e-04 -
root_mean_squared_error: 0.0219 - val_loss: 5.8126e-04 -
val_root_mean_squared_error: 0.0241
```

```
Epoch 18/120
845/845 [==============================] - 5s 6ms/step - loss: 4.7716e-04 -
root_mean_squared_error: 0.0218 - val_loss: 6.0312e-04 -
val_root_mean_squared_error: 0.0246
Epoch 19/120
845/845 [==============================] - 5s 6ms/step - loss: 4.7649e-04 -
root_mean_squared_error: 0.0218 - val_loss: 5.5513e-04 -
val_root_mean_squared_error: 0.0236
Epoch 20/120
845/845 [==============================] - 5s 6ms/step - loss: 4.6142e-04 -
root_mean_squared_error: 0.0215 - val_loss: 5.9848e-04 -
val_root_mean_squared_error: 0.0245
Epoch 21/120
845/845 [==============================] - 5s 6ms/step - loss: 4.4911e-04 -
root_mean_squared_error: 0.0212 - val_loss: 5.9896e-04 -
val_root_mean_squared_error: 0.0245
Epoch 22/120
845/845 [==============================] - 5s 6ms/step - loss: 4.5364e-04 -
root_mean_squared_error: 0.0213 - val_loss: 5.9325e-04 -
val_root_mean_squared_error: 0.0244
Epoch 23/120
845/845 [==============================] - 5s 6ms/step - loss: 4.4624e-04 -
root_mean_squared_error: 0.0211 - val_loss: 6.8024e-04 -
val_root_mean_squared_error: 0.0261
Epoch 24/120
845/845 [==============================] - 5s 6ms/step - loss: 4.4151e-04 -
root_mean_squared_error: 0.0210 - val_loss: 7.6999e-04 -
val_root_mean_squared_error: 0.0278
Epoch 25/120
845/845 [==============================] - 5s 6ms/step - loss: 4.3878e-04 -
root_mean_squared_error: 0.0209 - val_loss: 5.8290e-04 -
val_root_mean_squared_error: 0.0242
Epoch 26/120
845/845 [==============================] - 5s 6ms/step - loss: 4.3937e-04 -
root_mean_squared_error: 0.0210 - val_loss: 6.6545e-04 -
val_root_mean_squared_error: 0.0258
Epoch 27/120
845/845 [==============================] - 5s 6ms/step - loss: 4.3266e-04 -
root_mean_squared_error: 0.0208 - val_loss: 6.0309e-04 -
val_root_mean_squared_error: 0.0246
Epoch 28/120
845/845 [==============================] - 5s 6ms/step - loss: 4.2452e-04 -
root_mean_squared_error: 0.0206 - val_loss: 7.2058e-04 -
val_root_mean_squared_error: 0.0269
Epoch 29/120
845/845 [==============================] - 5s 6ms/step - loss: 4.1941e-04 -
root_mean_squared_error: 0.0205 - val_loss: 5.8634e-04 -
val_root_mean_squared_error: 0.0242
```

[107]:
```
plot_model_rmse_and_loss(history)
```





[108]:
```
multivariate_cnn_lstm = tf.keras.models.load_model('multivariate_cnn_lstm.h5')

forecast = multivariate_cnn_lstm.predict(X_test)
multivariate_cnn_lstm_forecast = scaler_y.inverse_transform(forecast)

rmse_mult_cnn_lstm = sqrt(mean_squared_error(y_test_inv,
                                             multivariate_cnn_lstm_forecast))
print('RMSE of hour-ahead electricity price multivariate CNN-`LSTM forecast: {}'
      .format(round(rmse_mult_cnn_lstm, 3)))
```

```
RMSE of hour-ahead electricity price multivariate CNN-`LSTM forecast: 2.309
```

[109]:
```
X_test.shape
```

[109]:
```
(3984, 24, 17)
```

6.7 4.7. Time Distributed MLP

[110]:
```
tf.keras.backend.clear_session()

multivariate_mlp = tf.keras.models.Sequential([
    TimeDistributed(Dense(200, activation='relu'),
                    input_shape=input_shape),
    TimeDistributed(Dense(150, activation='relu')),
    TimeDistributed(Dense(100, activation='relu')),
    TimeDistributed(Dense(50, activation='relu')),
    Flatten(),
    Dense(150, activation='relu'),
    Dropout(0.1),
    Dense(1)
])

model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
                   'multivariate_mlp.h5', save_best_only=True)
optimizer = tf.keras.optimizers.Adam(lr=2e-3, amsgrad=True)

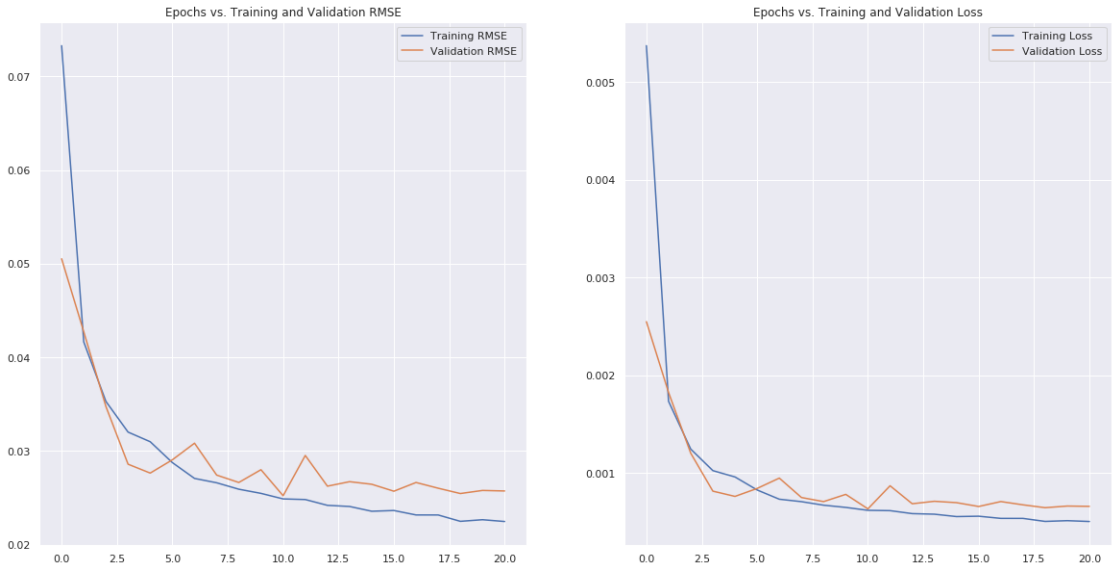
multivariate_mlp.compile(loss=loss,
                         optimizer=optimizer,
                         metrics=metric)
```

[111]:
```
history = multivariate_mlp.fit(train, epochs=120,
                               validation_data=validation,
                               callbacks=[early_stopping,
                                          model_checkpoint])
```

```
Epoch 1/120
845/845 [==============================] - 5s 6ms/step - loss: 0.0054 -
root_mean_squared_error: 0.0733 - val_loss: 0.0000e+00 -
val_root_mean_squared_error: 0.0000e+00
Epoch 2/120
845/845 [==============================] - 4s 4ms/step - loss: 0.0017 -
root_mean_squared_error: 0.0416 - val_loss: 0.0018 -
val_root_mean_squared_error: 0.0427
Epoch 3/120
845/845 [==============================] - 3s 4ms/step - loss: 0.0012 -
root_mean_squared_error: 0.0353 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0348
Epoch 4/120
845/845 [==============================] - 3s 4ms/step - loss: 0.0010 -
root_mean_squared_error: 0.0320 - val_loss: 8.1535e-04 -
val_root_mean_squared_error: 0.0286
Epoch 5/120
845/845 [==============================] - 3s 4ms/step - loss: 9.6047e-04 -
root_mean_squared_error: 0.0310 - val_loss: 7.6166e-04 -
val_root_mean_squared_error: 0.0276
Epoch 6/120
845/845 [==============================] - 3s 4ms/step - loss: 8.2742e-04 -
root_mean_squared_error: 0.0288 - val_loss: 8.4305e-04 -
val_root_mean_squared_error: 0.0291
Epoch 7/120
845/845 [==============================] - 3s 4ms/step - loss: 7.3305e-04 -
root_mean_squared_error: 0.0271 - val_loss: 9.4873e-04 -
val_root_mean_squared_error: 0.0308
Epoch 8/120
845/845 [==============================] - 3s 4ms/step - loss: 7.0790e-04 -
root_mean_squared_error: 0.0266 - val_loss: 7.5021e-04 -
val_root_mean_squared_error: 0.0274
Epoch 9/120
845/845 [==============================] - 4s 4ms/step - loss: 6.7213e-04 -
root_mean_squared_error: 0.0259 - val_loss: 7.0850e-04 -
val_root_mean_squared_error: 0.0266
Epoch 10/120
845/845 [==============================] - 4s 4ms/step - loss: 6.4921e-04 -
root_mean_squared_error: 0.0255 - val_loss: 7.8233e-04 -
val_root_mean_squared_error: 0.0280
Epoch 11/120
845/845 [==============================] - 3s 4ms/step - loss: 6.1941e-04 -
root_mean_squared_error: 0.0249 - val_loss: 6.3422e-04 -
val_root_mean_squared_error: 0.0252
Epoch 12/120
845/845 [==============================] - 3s 4ms/step - loss: 6.1593e-04 -
root_mean_squared_error: 0.0248 - val_loss: 8.7037e-04 -
val_root_mean_squared_error: 0.0295
Epoch 13/120
845/845 [==============================] - 3s 4ms/step - loss: 5.8559e-04 -
root_mean_squared_error: 0.0242 - val_loss: 6.8652e-04 -
val_root_mean_squared_error: 0.0262
Epoch 14/120
845/845 [==============================] - 3s 4ms/step - loss: 5.7981e-04 -
root_mean_squared_error: 0.0241 - val_loss: 7.1178e-04 -
val_root_mean_squared_error: 0.0267
Epoch 15/120
845/845 [==============================] - 3s 4ms/step - loss: 5.5549e-04 -
root_mean_squared_error: 0.0236 - val_loss: 6.9774e-04 -
val_root_mean_squared_error: 0.0264
Epoch 16/120
845/845 [==============================] - 3s 4ms/step - loss: 5.5918e-04 -
root_mean_squared_error: 0.0236 - val_loss: 6.5847e-04 -
```

```
val_root_mean_squared_error: 0.0257
Epoch 17/120
845/845 [==============================] - 3s 4ms/step - loss: 5.3697e-04 -
root_mean_squared_error: 0.0232 - val_loss: 7.0896e-04 -
val_root_mean_squared_error: 0.0266
Epoch 18/120
845/845 [==============================] - 3s 4ms/step - loss: 5.3679e-04 -
root_mean_squared_error: 0.0232 - val_loss: 6.7541e-04 -
val_root_mean_squared_error: 0.0260
Epoch 19/120
845/845 [==============================] - 3s 4ms/step - loss: 5.0573e-04 -
root_mean_squared_error: 0.0225 - val_loss: 6.4615e-04 -
val_root_mean_squared_error: 0.0255
Epoch 20/120
845/845 [==============================] - 4s 4ms/step - loss: 5.1361e-04 -
root_mean_squared_error: 0.0227 - val_loss: 6.6299e-04 -
val_root_mean_squared_error: 0.0258
Epoch 21/120
845/845 [==============================] - 3s 4ms/step - loss: 5.0471e-04 -
root_mean_squared_error: 0.0225 - val_loss: 6.5991e-04 -
val_root_mean_squared_error: 0.0257
```

[112]:
```
plot_model_rmse_and_loss(history)
```



[113]:
```
multivariate_mlp = tf.keras.models.load_model('multivariate_mlp.h5')

forecast = multivariate_mlp.predict(X_test)
multivariate_mlp_forecast = scaler_y.inverse_transform(forecast)

rmse_mult_mlp = sqrt(mean_squared_error(y_test_inv,
                                        multivariate_mlp_forecast))
print('RMSE of hour-ahead electricity price multivariate MLP forecast: {}'
      .format(round(rmse_mult_mlp, 3)))
```

```
RMSE of hour-ahead electricity price multivariate MLP forecast: 2.331
```

6.8 4.8. Encoder-Decoder

[114]:
```
tf.keras.backend.clear_session()

encoder_decoder = tf.keras.models.Sequential([
    LSTM(50, activation='relu', input_shape=input_shape),
    RepeatVector(past_history),
    LSTM(50, activation='relu', return_sequences=True),
    TimeDistributed(Dense(50, activation='relu')),
    Flatten(),
    Dense(25, activation='relu'),
    Dense(1)
])



model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
                   'encoder_decoder.h5', save_best_only=True)

optimizer = tf.keras.optimizers.Adam(lr=1e-3, amsgrad=True)

encoder_decoder.compile(loss=loss,
                    optimizer=optimizer,
                    metrics=metric)
```

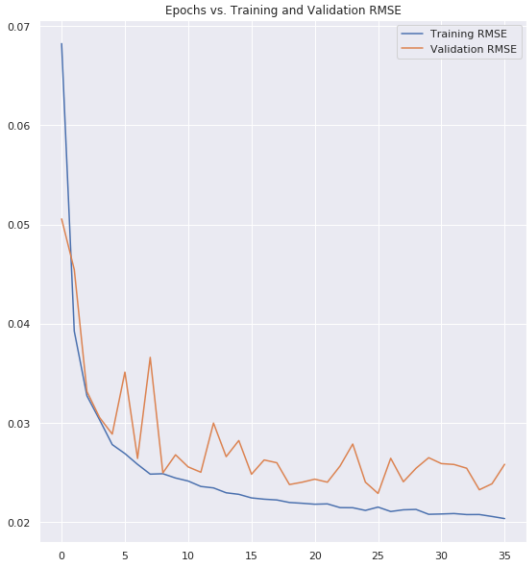
[115]:
```
history = encoder_decoder.fit(train, epochs=50,
                              validation_data=validation,
                              callbacks=[early_stopping,
                                         model_checkpoint])
```

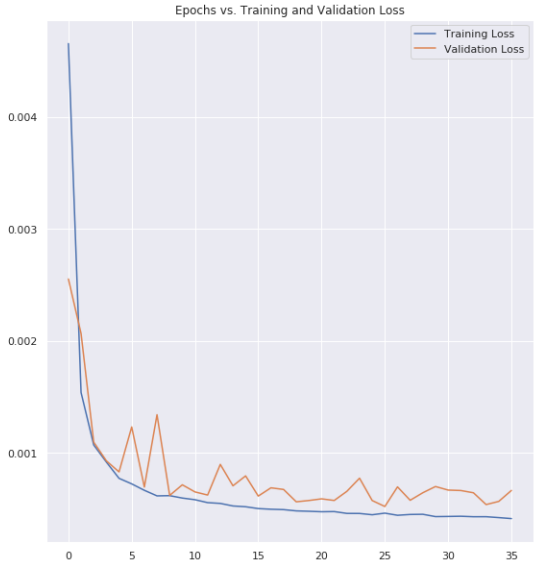
```
Epoch 1/50
845/845 [==============================] - 47s 56ms/step - loss: 0.0047 -
root_mean_squared_error: 0.0682 - val_loss: 0.0000e+00 -
val_root_mean_squared_error: 0.0000e+00
Epoch 2/50
845/845 [==============================] - 44s 52ms/step - loss: 0.0015 -
root_mean_squared_error: 0.0393 - val_loss: 0.0021 -
val_root_mean_squared_error: 0.0455
Epoch 3/50
845/845 [==============================] - 44s 52ms/step - loss: 0.0011 -
root_mean_squared_error: 0.0328 - val_loss: 0.0011 -
val_root_mean_squared_error: 0.0332
Epoch 4/50
845/845 [==============================] - 43s 51ms/step - loss: 9.2001e-04 -
root_mean_squared_error: 0.0303 - val_loss: 9.3069e-04 -
val_root_mean_squared_error: 0.0306
Epoch 5/50
845/845 [==============================] - 44s 53ms/step - loss: 7.7430e-04 -
root_mean_squared_error: 0.0278 - val_loss: 8.3191e-04 -
val_root_mean_squared_error: 0.0289
Epoch 6/50
845/845 [==============================] - 43s 51ms/step - loss: 7.2523e-04 -
root_mean_squared_error: 0.0269 - val_loss: 0.0012 -
val_root_mean_squared_error: 0.0351
Epoch 7/50
845/845 [==============================] - 44s 52ms/step - loss: 6.6786e-04 -
root_mean_squared_error: 0.0258 - val_loss: 6.9690e-04 -
val_root_mean_squared_error: 0.0264
Epoch 8/50
845/845 [==============================] - 43s 51ms/step - loss: 6.1771e-04 -
root_mean_squared_error: 0.0249 - val_loss: 0.0013 -
val_root_mean_squared_error: 0.0366
Epoch 9/50
845/845 [==============================] - 44s 52ms/step - loss: 6.2022e-04 -
root_mean_squared_error: 0.0249 - val_loss: 6.2217e-04 -
val_root_mean_squared_error: 0.0250
Epoch 10/50
```

```
845/845 [==============================] - 44s 52ms/step - loss: 5.9876e-04 -
root_mean_squared_error: 0.0245 - val_loss: 7.1680e-04 -
val_root_mean_squared_error: 0.0268
Epoch 11/50
845/845 [==============================] - 45s 53ms/step - loss: 5.8381e-04 -
root_mean_squared_error: 0.0242 - val_loss: 6.5328e-04 -
val_root_mean_squared_error: 0.0256
Epoch 12/50
845/845 [==============================] - 44s 52ms/step - loss: 5.5685e-04 -
root_mean_squared_error: 0.0236 - val_loss: 6.2545e-04 -
val_root_mean_squared_error: 0.0251
Epoch 13/50
845/845 [==============================] - 44s 52ms/step - loss: 5.5097e-04 -
root_mean_squared_error: 0.0235 - val_loss: 8.9911e-04 -
val_root_mean_squared_error: 0.0300
Epoch 14/50
845/845 [==============================] - 43s 51ms/step - loss: 5.2806e-04 -
root_mean_squared_error: 0.0230 - val_loss: 7.0799e-04 -
val_root_mean_squared_error: 0.0266
Epoch 15/50
845/845 [==============================] - 43s 51ms/step - loss: 5.2100e-04 -
root_mean_squared_error: 0.0228 - val_loss: 7.9641e-04 -
val_root_mean_squared_error: 0.0282
Epoch 16/50
845/845 [==============================] - 45s 53ms/step - loss: 5.0460e-04 -
root_mean_squared_error: 0.0225 - val_loss: 6.1626e-04 -
val_root_mean_squared_error: 0.0248
Epoch 17/50
845/845 [==============================] - 43s 51ms/step - loss: 4.9873e-04 -
root_mean_squared_error: 0.0223 - val_loss: 6.8998e-04 -
val_root_mean_squared_error: 0.0263
Epoch 18/50
845/845 [==============================] - 44s 52ms/step - loss: 4.9519e-04 -
root_mean_squared_error: 0.0223 - val_loss: 6.7537e-04 -
val_root_mean_squared_error: 0.0260
Epoch 19/50
845/845 [==============================] - 44s 52ms/step - loss: 4.8417e-04 -
root_mean_squared_error: 0.0220 - val_loss: 5.6517e-04 -
val_root_mean_squared_error: 0.0238
Epoch 20/50
845/845 [==============================] - 45s 53ms/step - loss: 4.8031e-04 -
root_mean_squared_error: 0.0219 - val_loss: 5.7670e-04 -
val_root_mean_squared_error: 0.0240
Epoch 21/50
845/845 [==============================] - 45s 53ms/step - loss: 4.7653e-04 -
root_mean_squared_error: 0.0218 - val_loss: 5.9155e-04 -
val_root_mean_squared_error: 0.0243
Epoch 22/50
845/845 [==============================] - 45s 53ms/step - loss: 4.7777e-04 -
root_mean_squared_error: 0.0219 - val_loss: 5.7702e-04 -
val_root_mean_squared_error: 0.0240
Epoch 23/50
845/845 [==============================] - 44s 52ms/step - loss: 4.6138e-04 -
root_mean_squared_error: 0.0215 - val_loss: 6.5861e-04 -
val_root_mean_squared_error: 0.0257
Epoch 24/50
845/845 [==============================] - 43s 51ms/step - loss: 4.6109e-04 -
root_mean_squared_error: 0.0215 - val_loss: 7.7580e-04 -
val_root_mean_squared_error: 0.0279
Epoch 25/50
845/845 [==============================] - 44s 52ms/step - loss: 4.4951e-04 -
root_mean_squared_error: 0.0212 - val_loss: 5.7650e-04 -
val_root_mean_squared_error: 0.0240
Epoch 26/50
845/845 [==============================] - 44s 52ms/step - loss: 4.6367e-04 -
root_mean_squared_error: 0.0215 - val_loss: 5.2331e-04 -
val_root_mean_squared_error: 0.0229
Epoch 27/50
```

```
 90/845 [==>…] - ETA: 35s - loss: 6.8752e-04 -
root_mean_squared_error: 0.0262
```

[116]:
```
plot_model_rmse_and_loss(history)
```





[117]:
```
encoder_decoder = tf.keras.models.load_model('encoder_decoder.h5')

forecast = encoder_decoder.predict(X_test)
encoder_decoder_forecast = scaler_y.inverse_transform(forecast)

rmse_encoder_decoder = sqrt(mean_squared_error(y_test_inv,
                                               encoder_decoder_forecast))
print('RMSE of hour-ahead electricity price Encoder-Decoder forecast: {}'
      .format(round(rmse_encoder_decoder, 3)))
```

```
RMSE of hour-ahead electricity price Encoder-Decoder forecast: 2.296
```

7 5. Results

8 6. References

- [1] Ortiz, M.; Ukar, O.; Azevedo, F. and Múgica, A. (2016). Price forecasting and validation in the Spanish electricity market using forecasts as input data, International Journal of Electrical Power & Energy Systems , 77 : 123-127.
- [2] https://en.wikipedia.org/wiki/Orders_of_magnitude_(pressure)
- [3] https://en.wikipedia.org/wiki/Orders_of_magnitude_(speed)
- [4] https://en.wikipedia.org/wiki/Fujita_scale
- [5] https://en.wikipedia.org/wiki/List_of_European_tornadoes_and_tornado_outbreaks
- [6] https://en.wikipedia.org/wiki/List_of_atmospheric_pressure_records_in_Europe#Iberia
- [7] https://en.wikipedia.org/wiki/List_of_metropolitan_areas_in_Spain

** Thank you for your time! If you like this notebook, **PLEASE UPVOTE!** Do not hestitate to share your thoughts and do not forget to check again, as the notebook will be updated soon! **